



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ &
ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Βάσεις Δεδομένων: Εξαμηνιαία Εργασία 2025-2026
Σύστημα αποθήκευσης και διαχείρισης των πληροφοριών για το
Γενικό Νοσοκομείο «Υγειόπολης»

Μιχάλης Ψυχής 03122094
Νικήτας Νικολόπουλος 03123192
Λουκάς Φραγκούλης 03123835

ΠΑΡΑΔΟΧΕΣ/ΥΠΟΘΕΣΕΙΣ (βρίσκονται και στο README.md)

1. Διαχείριση κατηγορικών πεδίων (lookup tables αντί ENUM)

Η εκφώνηση απαγορεύει ρητά τη χρήση τύπων ENUM, JSON, array και XML. Όπου το πεδίο έχει σταθερό λεξιλόγιο (π.χ. φύλο, ασφαλιστικός φορέας, βαθμίδα ιατρού, επίπεδο επείγοντος, τύπος κλίνης, κατάσταση κλίνης, slot βάρδιας, κατηγορία επέμβασης κ.λπ.) μοντελοποιείται ως ξεχωριστός πίνακας αναφοράς (lookup) και η σχετική στήλη της βασικής οντότητας τον αναφέρει μέσω foreign key. Έτσι εξασφαλίζεται referential integrity, αποτρέπονται typos, και επιτρέπεται η μελλοντική επέκταση χωρίς ALTER TABLE.

2. Ιεραρχία προσωπικού (ISA) — table-per-subtype

Η ISA ιεραρχία staff to {doctors, nurses, admin_staff} υλοποιείται ως table-per-subtype: ένας υπερπίνακας staff και τρεις υποπίνακες με κοινό κλειδί (AMKA). Κάθε υπότυπος αναφέρει τον γονέα μέσω FK με ON DELETE CASCADE. Η εξειδίκευση θεωρείται disjoint και total κάθε εγγραφή στον staff ανήκει ακριβώς σε έναν υπότυπο. Η ένδειξη υποτύπου διατηρείται στη στήλη staff.staff_type (FK προς lookup) ως discriminator για ευκολία σε queries όπως το Q8.

3. Ηλικία ως παράγωγο γνώρισμα από ημερομηνία γέννησης

Η εκφώνηση αναφέρει την «ηλικία» τόσο για το προσωπικό όσο και για τους ασθενείς. Επιλέχθηκε να αποθηκευτεί η ημερομηνία γέννησης (date_of_birth) και η ηλικία να είναι virtual column υπολογιζόμενη με TIMESTAMPDIFF. Αυτό αποτρέπει την αλλοίωση της πληροφορίας με τον χρόνο και αποτελεί κανονική (1NF/3NF-φιλική) λύση.

4. Μετονομασία «Νοσηλεία» to admissions

Η οντότητα που η εκφώνηση ονομάζει «νοσηλεία» αντιστοιχίζεται στον πίνακα admissions. Μια εγγραφή admissions αντιστοιχεί σε ένα συνεχές διάστημα εισαγωγής σε συγκεκριμένη κλίνη και τμήμα.

5. Μοντελοποίηση κόστους νοσηλείας (KEN)

Το κόστος μιας νοσηλείας ακολουθεί αυστηρά το σύστημα KEN της εκφώνησης: το base_cost καθορίζεται από τον αντίστοιχο κωδικό KEN, ενώ το extra_cost υπολογίζεται μόνο όταν η πραγματική παραμονή ξεπεράσει τη Μέση Διάρκεια Νοσηλείας (ΜΔΝ), με βάση το daily_surcharge_rate του κωδικού. Το total_cost είναι virtual column ίσο με base_cost + extra_cost. Τα κόστη εργαστηριακών εξετάσεων (lab_exams.cost) και ιατρικών πράξεων (procedure_executions.actual_cost) αποθηκεύονται ξεχωριστά ανά συμβάν για εσωτερική λογιστική, αλλά δεν προστίθενται στο συνολικό κόστος νοσηλείας. Αυτό συμφωνεί με τη φιλοσοφία του KEN ως bundled price και με τη διατύπωση του ερωτήματος Q1, που ρητά ζητά διαχωρισμό μόνο σε «βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ».

6. bed_count ως αποθηκευμένο πεδίο (όχι παράγωγο)

Η εκφώνηση παραθέτει τον «αριθμό κλινών» ως γνώρισμα του τμήματος. Παρότι θα μπορούσε να υπολογίζεται με COUNT(*) πάνω στον πίνακα beds, επιλέχθηκε να αποθηκευτεί ως στήλη του

departments και να διατηρείται συνεπής μέσω trigger. Η επιλογή αυτή κρατά την οντότητα departments όσο πιο κοντά στη λεκτική διατύπωση της εκφώνησης, με αμελητέο κόστος συντήρησης για ένα μέγεθος της τάξης των δεκαπέντε τμημάτων.

7. Triage ως ξεχωριστή οντότητα, σύνδεση 1:1 με admissions

Η διαλογή (triage) μοντελοποιείται ως αυτόνομη οντότητα και όχι ως πεδία του admissions, επειδή κατά την εκφώνηση ένας ασθενής μπορεί να περάσει από triage χωρίς να καταλήξει σε νοσηλεία (αποχωρεί με οδηγίες). Όταν η διαλογή οδηγεί σε νοσηλεία, η εγγραφή triages αναφέρει την αντίστοιχη admissions μέσω FK με UNIQUE constraint, εξασφαλίζοντας σχέση 1:1.

Επιπλέον, το πεδίο minutes_waited αποθηκεύεται ρητά (αντί να υπολογίζεται από τα arrival_time και triage_time) ως denormalization για επιτάχυνση του Q15 («μέσος χρόνος αναμονής ανά επίπεδο επείγοντος»), και επειδή η έννοια του «χρόνου αναμονής» μπορεί στο μέλλον να ορίζεται διαφορετικά (π.χ. από την άφιξη μέχρι την παραπομπή).

8. Αλλεργίες σε δραστικές ουσίες (όχι σε φάρμακα)

Σύμφωνα με την εκφώνηση, οι αλλεργίες καταγράφονται σε επίπεδο δραστικής ουσίας (active_substances) και όχι φαρμάκου. Ένα φάρμακο μπορεί να περιέχει πολλές δραστικές ουσίες, και cross-reactions μεταξύ διαφορετικών φαρμάκων που μοιράζονται την ίδια δραστική ουσία πρέπει να ανιχνεύονται κατά τη συνταγογράφηση. Η σχέση patients with active_substances υλοποιείται ως M:N junction (patient_allergies) χωρίς επιπλέον γνωρίσματα.

9. Εικόνες / επιλεκτική κάλυψη οντοτήτων

Η εκφώνηση αναφέρει ότι «όλες οι οντότητες» θα συνοδεύονται από εικόνες στην προβλεπόμενη ιστοσελίδα. Πρακτικά, μοντελοποιήσαμε πίνακες εικόνων μόνο για τις showcase οντότητες με δημόσιο ενδιαφέρον: department_images, doctor_images, drug_images, operating_room_images. Οντότητες όπως ασθενείς και βάρδιες εξαιρέθηκαν. Η ιατρική πράξη (medical_procedures) επίσης εξαιρέθηκε, καθώς αποτελεί αφηρημένο κωδικό από τον κατάλογο και όχι φυσική οντότητα.

10. Ένας ιατρός - μία κύρια ειδικότητα

Η εκφώνηση χρησιμοποιεί την «ειδικότητα» στον ενικό. Στο σχεδιασμό μας, κάθε ιατρός έχει ακριβώς μία κύρια ειδικότητα, η οποία αναφέρεται από τη στήλη doctors.specialty ως FK προς τον lookup specialties. Πολλαπλές ειδικότητες δεν υποστηρίζονται και δεν θεωρούνται απαραίτητες για κανένα από τα ερωτήματα της εκφώνησης, συμπεριλαμβανομένου του Q2.

11. Σχέσεις προσωπικού με τμήμα

Ιατροί προς τμήματα: M:N μέσω doctor_departments, δηλαδή ένας ιατρός μπορεί να ανήκει σε πολλά τμήματα. Νοσηλευτές προς τμήμα: 1:N (nurses.dept_id), ανήκουν σε ακριβώς ένα τμήμα. Διοικητικό προσωπικό προς τμήμα: 1:N (admin_staff.dept_id), ανήκουν σε ακριβώς ένα τμήμα.

Διευθυντής τμήματος: 1:1 (departments.director_amka FK προς doctors, nullable για περιπτώσεις προσωρινής έλλειψης).

12. Σχέση relationship_to_patient σε emergency contacts

Το πεδίο που περιγράφει τη σχέση του οικείου ατόμου με τον ασθενή (π.χ. «μητέρα», «σύζυγος», «φίλος») διατηρείται ως ελεύθερο κείμενο (VARCHAR), παρά τη γενική πολιτική του σχήματος για lookup tables. Η διακύμανση είναι πολύ μεγάλη για να καταγραφεί εξαντλητικά, και η συγκεκριμένη πληροφορία δεν χρησιμοποιείται σε κανένα από τα ερωτήματα. Λοιπές παραδοχές κανονικοποίησης δεν παραβιάζονται.

13. Κατηγοριοποίηση KEN / ξεχωριστός lookup ken_categories

Πέραν του ίδιου του κωδικού KEN, οι κωδικοί ομαδοποιούνται σε κατηγορίες (γράμμα/πρόθεμα). Αντί να αποθηκευτεί το γράμμα ως ελεύθερο πεδίο στον πίνακα ken, μοντελοποιείται ως ξεχωριστός lookup ken_categories(category_letter, description) με αναφορά από ken.category_letter. Η ίδια λογική εφαρμόζεται και στα ICD-10 με τη στήλη icd10.category.

14. Διαχείριση πληρότητας πληροφορίας στη νοσηλεία

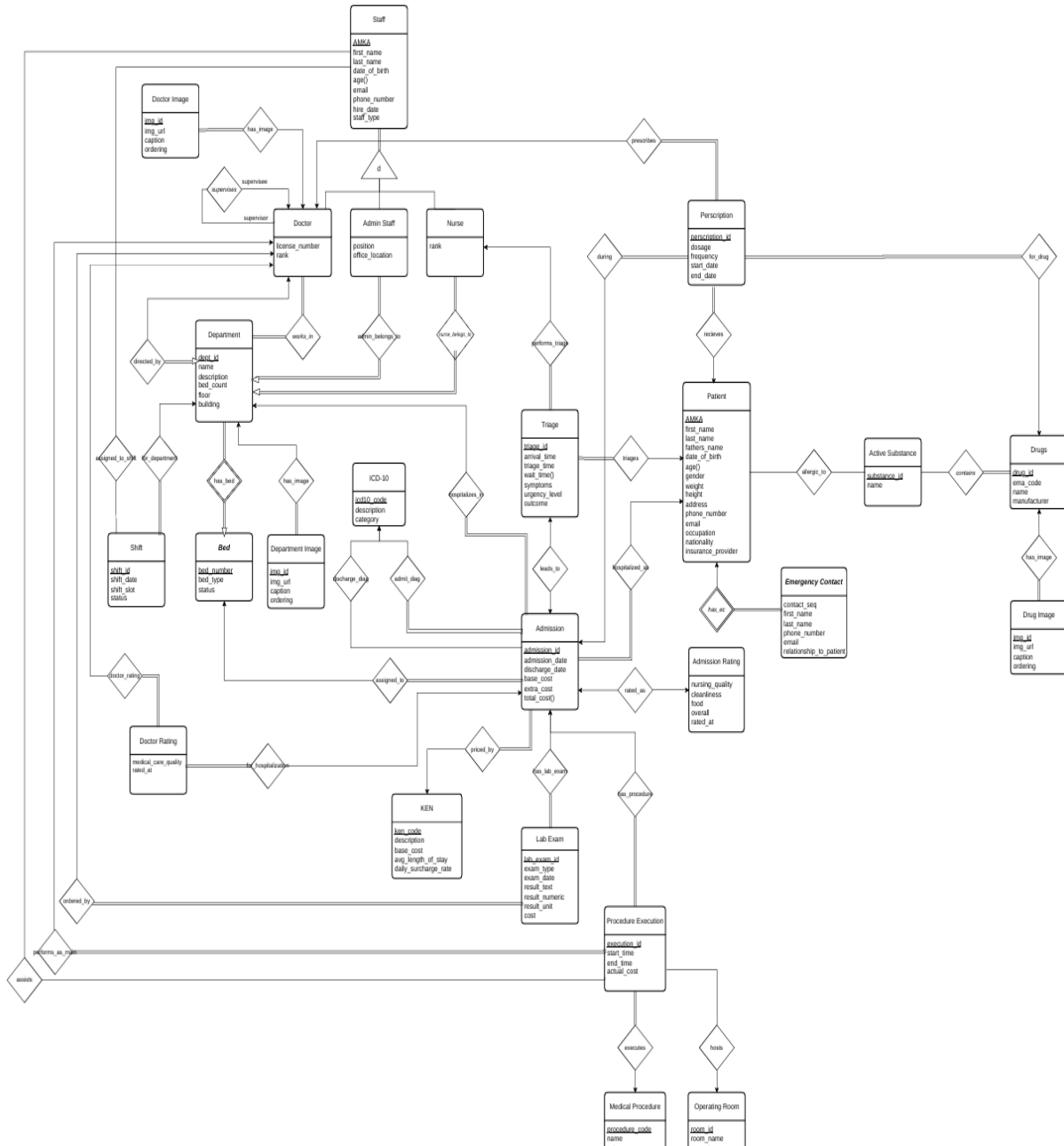
Κάποια πεδία της νοσηλείας συμπληρώνονται μετά την εισαγωγή του ασθενούς: discharge_date, discharge_diagnosis_code και η τελική τιμή του extra_cost. Αυτά μοντελοποιούνται ως nullable και ενημερώνονται κατά την έξοδο. Τα admission_date, admission_diagnosis_code, bed_id, department_id, ken_code και base_cost είναι υποχρεωτικά από τη στιγμή δημιουργίας της εγγραφής.

15. Reference data / εισαγωγή ως έχει από επίσημες πηγές

Τα δεδομένα αναφοράς (ICD-10, KEN, κατάλογος ιατρικών πράξεων, EMA Article 57) εισάγονται ως έχουν από τις αντίστοιχες επίσημες πηγές, σύμφωνα με τη ρητή απαίτηση της εκφώνησης. Δεν έχουν εφευρεθεί κωδικοί, ονόματα ή κατηγοριοποιήσεις. Οι λεπτομέρειες προ-επεξεργασίας και φόρτωσης δίνονται στην ενότητα «Φόρτωση δεδομένων».

1. Σχεδιασμός και υλοποίηση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (Entity - Relationship Diagram)



Σημασιολογικές Συμβάσεις του ER

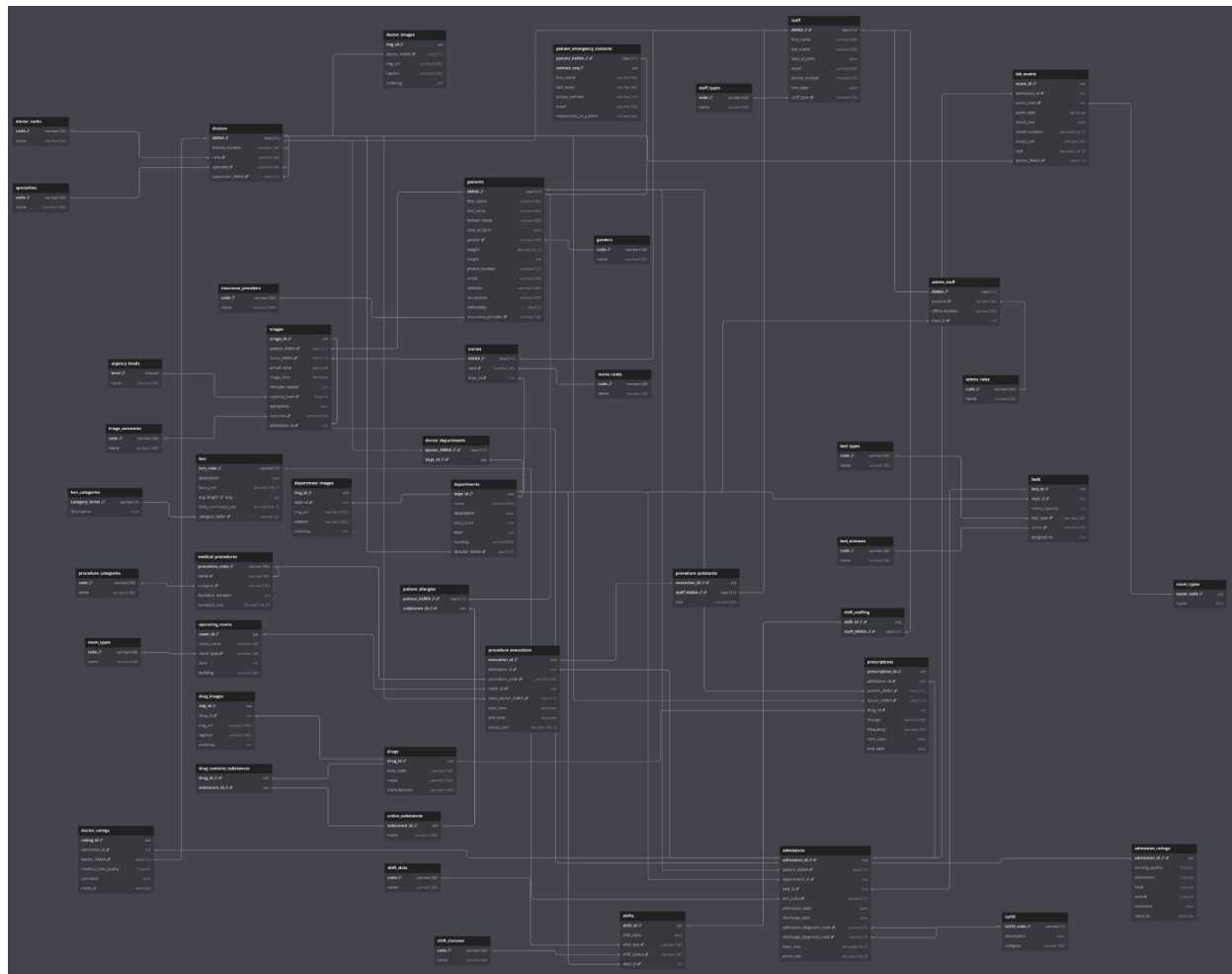
- Κάθε οντότητα αναπαρίσταται με ορθογώνιο που περιέχει τα γνωρίσματά της. Το όνομα της οντότητας γράφεται στην κορυφή του πλαισίου.
- Το Primary Key κάθε οντότητας εμφανίζεται με υπογράμμιση στο όνομα του γνωρίσματος (π.χ. ΑΜΚΑ στις οντότητες Staff και Patient, triage_id στην οντότητα Triage).
- Οι σχέσεις μεταξύ οντοτήτων συμβολίζονται με ρόμβους που αναγράφουν μονολεκτικά την εννοιολογική τους σύνδεση (π.χ. works_in, directed_by, prescribes, triages, leads_to).
- Η πληθικότητα (cardinality) των σχέσεων συμβολίζεται μέσω των γραμμών που ενώνουν την οντότητα με τον ρόμβο:
 1. Γραμμή χωρίς βέλος: σχέση many (η οντότητα συμμετέχει με πολλές εγγραφές)
 2. Γραμμή με βέλος που "δείχνει" προς την οντότητα: σχέση one (η οντότητα συμμετέχει με ακριβώς μία εγγραφή)

Άρα, για παράδειγμα, μια σχέση one-to-many συμβολίζεται με βέλος που "βλέπει" την οντότητα που είναι το one και απλή γραμμή χωρίς βέλος προς την οντότητα που είναι το many.
- Η συμμετοχή (participation) μιας οντότητας σε μια σχέση συμβολίζεται μέσω του πλήθους των γραμμών:
 1. Μονή γραμμή: μερική συμμετοχή (partial participation), όχι κάθε εγγραφή της οντότητας οφείλει να μετέχει στη σχέση.
 2. Διπλή γραμμή: ολική συμμετοχή (total participation), κάθε εγγραφή της οντότητας υποχρεωτικά μετέχει στη σχέση.
- Τα foreign keys δεν αναγράφονται ρητά στο ER, καθώς αποτελούν λεπτομέρεια υλοποίησης. Στο εννοιολογικό επίπεδο της Silberschatz notation οι FKs εκφράζονται έμμεσα μέσω των ιδίων των σχέσεων (ρόμβων) και της πληθικότητάς τους. Η ρητή αναγραφή τους θα εμφανιστεί στο σχεσιακό διάγραμμα και στο install.sql.
- Οι αδύναμες οντότητες (weak entities) είναι οντότητες που δεν μπορούν να υπάρχουν χωρίς την οντότητα στην οποία υπάγονται (identifying entity). Στο διάγραμμά μας, η οντότητα Emergency Contact είναι αδύναμη και εξαρτάται από την Patient. Οι αδύναμες οντότητες:
 1. Αναπαρίστανται με ορθογώνιο με italic όνομα και διπλό περίγραμμα.
 2. Έχουν partial key
 3. Ενώνονται με την identifying οντότητά τους μέσω διπλού ρόμβου (identifying relationship), στην περίπτωση μας ο ρόμβος has_ec.
- Η σχέση εξειδίκευσης (ISA hierarchy) μεταξύ Staff και των υποτύπων Doctor, Nurse και Admin Staff απεικονίζεται με τρίγωνο που περιέχει το γράμμα d (disjoint specialization. Κάθε εγγραφή Staff ανήκει σε ακριβώς έναν υπότυπο). Η γραμμή που συνδέει τη Staff με το τρίγωνο είναι διπλή, υποδηλώνοντας total specialization (κάθε εγγραφή Staff πρέπει υποχρεωτικά να ανήκει σε έναν από τους τρεις υπότυπους, δηλαδή δεν υπάρχει "γενικό" Staff χωρίς εξειδίκευση). Οι υπότυποι παραλείπουν τα κληρονομούμενα γνωρίσματα της Staff και αναγράφουν μόνο τα δικά τους πρόσθετα γνωρίσματα.
- Αναδρομικές σχέσεις (recursive relationships) όπου η ίδια οντότητα συμμετέχει και στις δύο άκρες, απεικονίζονται με τον ρόμβο να ενώνεται δύο φορές με την οντότητα, με ετικέτες ρόλων (role labels)

σε κάθε άκρη. Παράδειγμα: η σχέση supervises στην οντότητα Doctor χρησιμοποιεί τις ετικέτες supervisor και supervisee για να διακρίνει τους δύο ρόλους.

- Τα παράγωγα γνωρίσματα (derived attributes), δηλαδή τα γνωρίσματα που δεν αποθηκεύονται απευθείας αλλά υπολογίζονται από άλλα, αναγράφονται με παρενθέσεις στο τέλος του ονόματος (π.χ. age(), wait_time(), total_cost()). Το age υπολογίζεται από το date_of_birth, ενώ το wait_time από τις αρχικές χρονικές ενδείξεις της διαλογής.
- Τα κατηγορικά πεδία με σταθερό λεξιλόγιο (π.χ. staff_type, rank, bed_type, urgency_level, gender, insurance_provider) αναγράφονται στις αντίστοιχες οντότητες ως απλά γνωρίσματα. Στο σχήμα της βάσης υλοποιούνται ως ξεχωριστοί lookup tables με αναφορά μέσω foreign key, σύμφωνα με την Παραδοχή 1. Η λεπτομέρεια αυτή δεν αποτυπώνεται στο εννοιολογικό ER για λόγους αναγνωσιμότητας.
- Οι πίνακες εικόνων (Doctor Image, Department Image, Drug Image, Operating Room Image) αποτελούν δευτερεύουσες οντότητες που σχετίζονται με τη βασική τους οντότητα μέσω της σχέσης has_image. Κάθε βασική οντότητα μπορεί να έχει πολλές εικόνες (1:N), και κάθε εικόνα ανήκει υποχρεωτικά σε μία βασική οντότητα (διπλή γραμμή από την πλευρά της εικόνας).

1.2 Σχηματικό σχήμα (Relational Schema)



Από το σχεσιακό διάγραμμα μπορούμε να δούμε την συνολική δομή της βάσης, τους πίνακες και τις συσχετίσεις μεταξύ τους. Το διάγραμμα αυτό αποτελεί λεπτομερή εικόνα του `install.sql` αρχείου. Μπορούμε να εντοπίσουμε σημεία στο διάγραμμα για να καταλάβουμε πως αποτυπώνεται το `install.sql`. Για παράδειγμα:

```
CREATE TABLE patients (
    AMKA CHAR(11),
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    fathers_name VARCHAR(50),
    date_of_birth DATE,
    age INT AS (TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()))
VIRTUAL,
    gender VARCHAR(10) NOT NULL,
    weight DECIMAL(5,1),
    height INT,
    phone_number VARCHAR(15) UNIQUE,
    email VARCHAR(50) UNIQUE,
    address VARCHAR(255),
    occupation VARCHAR(50),
```

patients		
AMKA		CHAR(11)
first_name		VARCHAR(50) NN
last_name		VARCHAR(50) NN
fathers_name		VARCHAR(50)
date_of_birth		DATE
age		INT
gender		VARCHAR(10) NN
weight		DECIMAL(5, 1)
height		INT
phone_number		VARCHAR(15)
email		VARCHAR(50)
address		VARCHAR(255)
occupation		VARCHAR(50)
nationality		CHAR(2)
insurance_provider		VARCHAR(30)


```

nationality CHAR(2),
insurance_provider VARCHAR(30),

CONSTRAINT pk_patients PRIMARY KEY (AMKA),
CONSTRAINT fk_patient_gender FOREIGN KEY (gender) REFERENCES genders(code),
CONSTRAINT fk_patient_insurance FOREIGN KEY (insurance_provider) REFERENCES
insurance_providers(code)
);

```

Εδώ βλέπουμε την πλήρη αντιστοιχία του πίνακα των ασθενών μεταξύ του load.sql και του σχεσιακού διαγράμματος. Παρατηρούμε το πως ορίζεται ο τύπος και το μέγεθος κάθε γραμμής του πίνακα, καθώς επίσης και τα πεδία που έχουν τεθεί ως Not Null. Τα πεδία phone_number και email φέρουν τον περιορισμό UNIQUE, ο οποίος δεν επιτρέπει ίδιες εγγραφές. Λειτουργούν πρακτικά ως εναλλακτικά κλειδιά, προστατεύοντας την ποιότητα των καταχωρήσεων. Ενδιαφέρον έχουν και τα 3 constraints στο τέλος του table. Το πεδίο AMKA ορίζεται (μέσω του pk_patients) ως το primary κλειδί, ταυτοποιώντας την εγγραφή του κάθε ασθενή. Παράλληλα ορίζονται και δύο foreign keys για το φύλο του ασθενή και την ασφάλεια του. Τα constraints fk_patient_gender και fk_patient_insurance είναι θεμελιώδη για τη λογική της βάσης. Υποδεικνύουν ότι ο πίνακας δεν δέχεται ελεύθερο τυχαίο κείμενο για το φύλο ή τον ασφαλιστικό φορέα, αλλά η πληροφορία εξάγεται από τους αντίστοιχους πίνακες αναφοράς. Η δομή αυτών των foreign keys, φαίνεται παρακάτω:

```

CREATE TABLE genders (
    code VARCHAR(10),
    name VARCHAR(20) NOT NULL,
    CONSTRAINT pk_genders PRIMARY KEY (code)
);

CREATE TABLE insurance_providers (
    code VARCHAR(30),
    name VARCHAR(100) NOT NULL,
    CONSTRAINT pk_insurance_providers PRIMARY KEY (code)
);

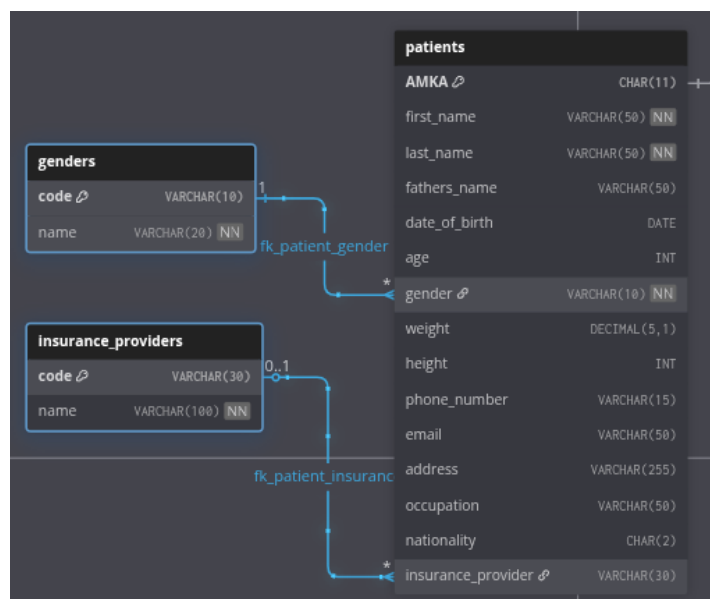
```

Όλες αυτές οι πληροφορίες φαίνονται και στο relation diagram. Συγκεκριμένα:

Δίπλα στο πεδίο AMKA του πίνακα patients, καθώς και στα πεδία code των πινάκων genders και insurance_providers, υπάρχει το μικρό εικονίδιο του κλειδιού. Αυτό υποδηλώνει το primary key

Δεξιά από τους τύπους δεδομένων σε πεδία όπως τα first_name, last_name και gender, βλέπουμε το χαρακτηριστικό ταμπελάκι NN (Not Null), ακριβώς όπως ορίστηκε στον κώδικα.

Τα εικονίδια με τον συνδετήρα δίπλα στα πεδία gender και insurance_provider του



πίνακα patients μαρτυρούν τον ρόλο τους ως ξένα κλειδιά.

Οι μπλε γραμμές που ενώνουν τους πίνακες φέρουν τα ακριβή ονόματα των constraints (fk_patient_gender και fk_patient_insurance). Το πιο ενδιαφέρον εδώ είναι η αποτύπωση της πληθικότητας στα άκρα των γραμμών:

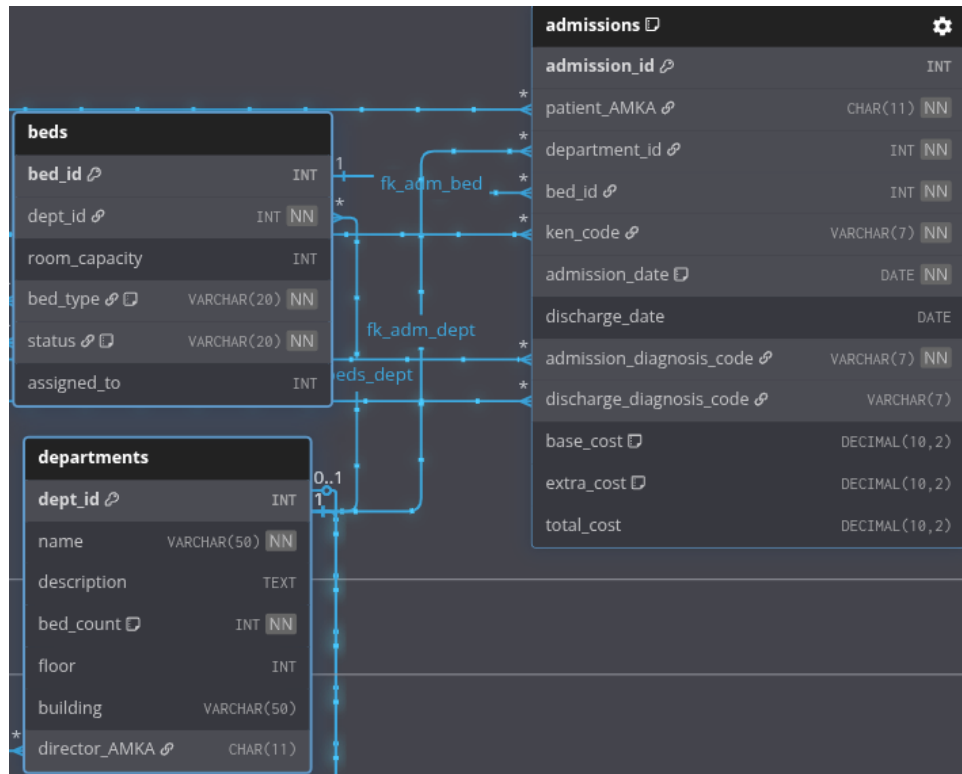
- Η σχέση με τα φύλα δείχνει 1 προς * (πολλά), υποδεικνύοντας ότι κάθε ασθενής πρέπει να έχει ένα (και μόνο ένα) καταχωρημένο φύλο.
- Η σχέση με τους παρόχους ασφάλισης δείχνει 0..1 προς * (πολλά). Αυτή είναι μια πολύ ωραία λεπτομέρεια, καθώς το 0..1 υποδηλώνει ότι ένας ασθενής μπορεί ενδεχομένως να μην έχει καταχωρημένο πάροχο ασφάλισης (κάτι που συνάδει με το γεγονός ότι το πεδίο insurance_provider δεν έχει περιορισμό NOT NULL στον κώδικα).

Γενικά η βάση σχεδιάστηκε με κάποιες συνειδητές αποφάσεις. Η πλειοψηφία του σχήματος βρίσκεται στην 3η Κανονική Μορφή, καθώς ο σχεδιασμός αυτός εξαλείφει τις περιττές επαναλήψεις και διατηρεί τις εξαρτήσεις των δεδομένων απόλυτα καθαρές. Κάθε χαρακτηριστικό εξαρτάται αποκλειστικά από το πρωτεύον κλειδί της οντότητάς του και από τίποτα άλλο. Για παράδειγμα, στοιχεία που αφορούν τα τμήματα ή τις ιατρικές ειδικότητες δεν επαναλαμβάνονται αυτούσια μέσα στους πίνακες των υπαλλήλων, αλλά διατηρούνται σε διακριτούς πίνακες αναφοράς.

Μια άλλη σχεδιαστική επιλογή που κάναμε ήταν η κληρονομικότητα των πινάκων. Για παράδειγμα η ύπαρξη ενός κεντρικού πίνακα staff και εξειδικευμένων πινάκων (doctors, nurses, admin_staff) οδηγεί στον διαχωρισμό των οντοτήτων. Το πιο ενδιαφέρον εδώ είναι η αποφυγή των πολλών κενών τιμών (NULL), καθώς ένας νοσηλευτής δεν χρειάζεται να έχει πεδίο ιατρικής ειδικότητας, παρόλο που αυτό απαιτεί να ενώσουμε τους πίνακες με JOIN για να ανακτήσουμε την πλήρη εικόνα ενός γιατρού.

Επίσης η χρήση της ιδιότητας VIRTUAL ήταν άλλη μια επιλογή μας. Σε πεδία όπως το age και το total_cost ο δυναμικός αυτός υπολογισμός εκείνη τη στιγμή χωρίς να καταναλώνουν χώρο στο δίσκο, διασφαλίζει ότι η ηλικία ή το κόστος είναι πάντα επικαιροποιημένα χωρίς να κινδυνεύει η συνέπεια των δεδομένων.

Υπάρχει ένα ξεκάθαρο σημείο όπου η 3η Κανονική Μορφή (3NF) δεν εφαρμόζεται από επιλογή μας. Ο Πίνακας admissions, περιέχει τα πεδία bed_id και department_id. Όμως, ένα κρεβάτι ανήκει ήδη σε ένα συγκεκριμένο τμήμα (dept_id). Επομένως, το department_id στο admissions είναι μια μεταβατική εξάρτηση (admission -> bed -> department). Δεδομένου ότι κάθε κλίνη υπάγεται σε συγκεκριμένο τμήμα, η αναγραφή του τμήματος απευθείας στην εισαγωγή δημιουργεί θεωρητικά τον κίνδυνο ασυνέπειας σε περίπτωση μεμονωμένων ενημερώσεων. Αυτή η προσέγγιση, ωστόσο, αποσκοπεί ξεκάθαρα στη βελτιστοποίηση της ταχύτητας των ερωτημάτων. Η αναζήτηση "Πόσες εισαγωγές είχε η Καρδιολογική;" θα απαιτούσε πάντα JOIN με τα beds. Βάζοντας το department_id κατευθείαν στο admissions, τα queries επιταχύνονται. Η ασυνέπεια έχει αποτραπεί μέσω των Triggers (check_admission_dept_matches_bed...) που επιβάλλουν τα δύο πεδία να συμβαδίζουν.



1.3 Indexes

Εισαγάγαμε τα ακόλουθα indexes στο db, λαμβάνοντας υπόψη τα queries που ζητούνται στην εκφώνηση.

```
-- Indexes

CREATE INDEX idx_adm_date ON admissions(admission_date);

CREATE INDEX idx_pe_start_time ON procedure_executions(start_time);

CREATE INDEX idx_shifts_date ON shifts(shift_date);

CREATE INDEX idx_triage_time ON triages(triage_time);

CREATE INDEX idx_icd10_category ON icd10(category);

CREATE INDEX idx_staff_last_name ON staff(last_name);
CREATE INDEX idx_patients_last_name ON patients(last_name);
CREATE INDEX idx_patients_dob ON patients(date_of_birth);
```

Οι στήλες που αποτελούν foreign keys δεν χρειάζεται να δηλωθούν ως indexes, καθώς η InnoDB δημιουργεί αυτόματα indexes για αυτές κατά τη δήλωση του FOREIGN KEY constraint.

1. Πίνακας admissions:

Index **idx_adm_date** στη στήλη admission_date. Χρησιμοποιείται για ομαδοποίηση των δεδομένων ανά έτος. Συγκεκριμένα, βελτιστοποιεί τα εξής ερωτήματα:

- Q01 και Q14: Όπου γίνεται φιλτράρισμα με βάση το έτος εισαγωγής.
- Q06: Όπου ταξινομούνται τα ιστορικά δεδομένα των ασθενών χρονολογικά.
- Q09: Όπου εξετάζεται η διάρκεια νοσηλείας των ασθενών μέσα στο ίδιο έτος.

2. Πίνακας procedure_executions:

Index **idx_pe_start_time** στη στήλη start_time. Χρησιμοποιείται για ομαδοποίηση των επεμβάσεων και εύρεση αυτών που έγιναν σε ένα συγκεκριμένο έτος (Q11).

3. Πίνακας shifts:

Index **idx_shifts_date** στη στήλη shift_date.

Παρόλο που σε αυτό τον πίνακα υπάρχει ήδη ένα UNIQUE constraint (dept_id, shift_date, shift_slot), το shift_date είναι δεύτερο πεδίο σε αυτό το tuple. Σε κάποια queries όμως απαιτείται αναζήτηση με βάση αποκλειστικά την ημερομηνία. Συγκεκριμένα, βελτιστοποιεί τα εξής queries:

- Q08: Για το γρήγορο φιλτράρισμα του προσωπικού που δεν έχει εφημερία σε μια συγκεκριμένη ημερομηνία.
- Q12: Για την εύρεση του απαιτούμενου προσωπικού ανά βάρδια μέσα στο εύρος μιας συγκεκριμένης εβδομάδας.

4. Πίνακας triages:

Index **idx_triage_time** στη στήλη `triage_time`: Στο Q15 χρειάζεται να φιλτράρουμε τα triages ανάλογα με το αν έχουν έγκυρη ώρα προσέλευσης.

5. Πίνακας icd10:

Index **idx_icd10_category** στη στήλη `category`:

- Στο Q14 απαιτείται η ομαδοποίηση των διαγνώσεων ανά κατηγορία. Επειδή το `icd10_category` δεν αποτελεί foreign key, το δηλώσαμε ως index.

6. Πίνακες staff και patients:

Δημιουργήθηκαν indexes σε στήλες που αποτελούν συχνά κριτήρια αναζήτησης (φίλτρα WHERE) από τους χρήστες ή τις εφαρμογές του νοσοκομείου. Συγκεκριμένα:

- **idx_staff_name** (πίνακας staff) στη στήλη `last_name`:

Q04: Επιτρέπει την ταχύτατη ανάκτηση των στοιχείων και των αξιολογήσεων ενός συγκεκριμένου ιατρού με βάση το επώνυμό του (WHERE `last_name` = 'Alexiou').

- **idx_patients_last_name** (πίνακας patients) στη στήλη `last_name`:

Q06: Επιταχύνει την εύρεση του ιστορικού νοσηλείων για έναν συγκεκριμένο ασθενή όταν η αναζήτηση γίνεται με το επώνυμό του (WHERE `last_name` = 'Papadopoulos').

- **idx_patients_dob** (πίνακας patients) στη στήλη `date_of_birth`:

Q05: Στο συγκεκριμένο ερώτημα ζητούνται οι νέοι ιατροί με ηλικία κάτω των 35 ετών. Δεδομένου ότι η ηλικία (`age`) είναι μια εικονική/υπολογιζόμενη στήλη (virtual column) που προκύπτει από την ημερομηνία γέννησης, το index στη στήλη `date_of_birth` επιτρέπει στη MySQL να φιλτράρει αποδοτικά τους ιατρούς χωρίς να χρειάζεται να σκανάρει ολόκληρο τον πίνακα (WHERE `age` < 35).

1.4 Triggers

Ακολουθεί η αναλυτική καταγραφή των triggers της βάσης δεδομένων, προσαρμοσμένη αποκλειστικά στο ύφος και τη δομή που ζήτησες:

beds_insert_trigger: Ενημερώνει αυτόματα τη χωρητικότητα των κλινών ενός τμήματος κατά την προσθήκη νέου εξοπλισμού. Πρακτικά ελέγχει και εκτελεί:

- Κάθε φορά που εισάγεται μια νέα εγγραφή κλίνης στον πίνακα `beds`, ο trigger αυξάνει αυτόματα κατά 1 την τιμή του πεδίου `bed_count` στον πίνακα `departments` για το αντίστοιχο τμήμα (`NEW.dept_id`).
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER INSERT trigger στον πίνακα `beds`.

beds_delete_trigger: Διατηρεί συγχρονισμένο το πλήθος των διαθέσιμων κρεβατιών ενός τμήματος μετά από αφαιρέσεις. Πρακτικά ελέγχει και εκτελεί:

- Κατά τη διαγραφή μιας κλίνης από τον πίνακα `beds`, ο trigger μειώνει αυτόματα κατά 1 την τιμή του πεδίου `bed_count` στον πίνακα `departments` για το τμήμα στο οποίο ανήκε (`OLD.dept_id`).
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER DELETE trigger στον πίνακα `beds`.

set_contact_seq_trigger: Παράγει αυτόματα έναν μοναδικό αύξοντα αριθμό για τις επαφές έκτακτης ανάγκης κάθε ασθενούς. Πρακτικά ελέγχει και εκτελεί:

- Πριν την εισαγωγή μιας νέας επαφής, υπολογίζει τη μέγιστη υπάρχουσα τιμή του πεδίου `contact_seq` για τον συγκεκριμένο ασθενή (`patient_amka`) και της προσθέτει 1. Αν δεν υπάρχει προηγούμενη επαφή, ξεκινάει αυτόματα από το 1.
- Τον καλεί η βάση δεδομένων αυτόματα ως `BEFORE INSERT trigger` στον πίνακα `patient_emergency_contacts`.

check_admission_dept_matches_bed_insert_trigger: Αποτρέπει τη λανθασμένη κατανομή ασθενών σε κλίνες εκτός του δηλωμένου τμήματος νοσηλείας. Πρακτικά ελέγχει εάν:

- Το τμήμα στο οποίο ανήκει η επιλεγμένη κλίνη (`bed_id`) ταυτίζεται με το τμήμα εισαγωγής (`department_id`) που δηλώνεται στη νέα νοσηλεία. Σε περίπτωση ασυμβατότητας, ακυρώνει την εγγραφή με μήνυμα σφάλματος.
- Τον καλεί η βάση δεδομένων αυτόματα ως `BEFORE INSERT trigger` στον πίνακα `admissions`.

check_admission_dept_matches_bed_update_trigger: Διασφαλίζει τη συμβατότητα κλίνης και τμήματος κατά την τροποποίηση των στοιχείων μιας νοσηλείας. Πρακτικά ελέγχει εάν:

- Μετά από μια αλλαγή στην κλίνη ή στο τμήμα μιας υπάρχουσας νοσηλείας, η νέα κλίνη εξακολουθεί να ανήκει στο σωστό τμήμα. Αν εντοπιστεί αναντιστοιχία, η τροποποίηση μπλοκάρεται.
- Τον καλεί η βάση δεδομένων αυτόματα ως `BEFORE UPDATE trigger` στον πίνακα `admissions`.

check_bed_overlap_insert_trigger: Προστατεύει τη διαθεσιμότητα των κλινών από διπλοκρατήσεις και ακατάλληλη χρήση. Πρακτικά ελέγχει εάν:

- Η επιλεγμένη κλίνη έχει κατάσταση «υπό συντήρηση», οπότε και απαγορεύει την εισαγωγή. Παράλληλα, ελέγχει αν η κλίνη είναι ήδη κατειλημμένη από άλλη ενεργή νοσηλεία κατά το ίδιο χρονικό διάστημα.
- Τον καλεί η βάση δεδομένων αυτόματα ως `BEFORE INSERT trigger` στον πίνακα `admissions`.

check_bed_overlap_update_trigger: Αποτρέπει χρονικές επικαλύψεις κατά την αλλαγή των ημερομηνιών ή της κλίνης μιας νοσηλείας. Πρακτικά ελέγχει εάν:

- Η τροποποίηση των ημερομηνιών εισαγωγής/εξιτηρίου ή η αλλαγή κλίνης δημιουργεί σύγκρουση με το χρονικό διάστημα μιας άλλης καταχωρημένης νοσηλείας για το ίδιο κρεβάτι.
- Τον καλεί η βάση δεδομένων αυτόματα ως `BEFORE UPDATE trigger` στον πίνακα `admissions`.

calculate_admission_base_cost_trigger: Αυτοματοποιεί την τιμολόγηση της νοσηλείας με βάση το εγκεκριμένο κρατικό πλαίσιο. Πρακτικά ελέγχει και εκτελεί:

- Κατά την εισαγωγή μιας νοσηλείας, ανακτά αυτόματα την τιμή `base_cost` από τον πίνακα των `KEN` (`ken`) με βάση τον κωδικό `ken_code`, την αποθηκεύει στη νέα εγγραφή και αρχικοποιεί το επιπλέον κόστος (`extra_cost`) στο μηδέν.

- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα admissions.

calculate_admission_extra_cost_trigger: Υπολογίζει δυναμικά τις οικονομικές επιβαρύνσεις υπέρβασης νοσηλείας και κλειδώνει το βασικό κόστος. Πρακτικά ελέγχει εάν:

- Γίνεται προσπάθεια χειροκίνητης αλλαγής του βασικού κόστους (το οποίο απαγορεύεται). Επιπλέον, κατά την καταχώρηση του εξιτηρίου, υπολογίζει αν οι ημέρες πραγματικής νοσηλείας ξεπερνούν τη μέση διάρκεια του ΚΕΝ και χρεώνει την αντίστοιχη ημερήσια επιβάρυνση.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα admissions.

set_bed_occupied_trigger: Δεσμεύει αυτόματα μια κλίνη στο σύστημα αμέσως μετά την ολοκλήρωση μιας εισαγωγής. Πρακτικά ελέγχει και εκτελεί:

- Μόλις οριστικοποιηθεί μια νέα εισαγωγή, αλλάζει αυτόματα την κατάσταση της επιλεγμένης κλίνης στον πίνακα beds σε 'κατειλημμένη' και ενημερώνει το πεδίο assigned_to με το ID της νοσηλείας.
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER INSERT trigger στον πίνακα admissions.

set_bed_free_trigger: Απελευθερώνει αυτόματα την κλίνη μόλις ο ασθενής λάβει το εξιτήριο του. Πρακτικά ελέγχει και εκτελεί:

- Όταν το πεδίο discharge_date μιας νοσηλείας αλλάζει από NULL σε συγκεκριμένη ημερομηνία, ο trigger μετατρέπει την κατάσταση της κλίνης σε 'διαθέσιμη' και μηδενίζει το πεδίο assigned_to.
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER UPDATE trigger στον πίνακα admissions.

doctor_rank_insert_check_trigger: Επιβάλλει τους κανόνες ιεραρχίας και εποπτείας κατά την εγγραφή νέων γιατρών. Πρακτικά ελέγχει εάν:

- Ένας γιατρός που εισάγεται με τον βαθμό 'Ειδικευόμενος' στερείται επόπτη ή αν ένας γιατρός με τον βαθμό 'Διευθυντής' έχει δηλωμένο επόπτη, μπλοκάροντας την εγγραφή σε κάθε τέτοια περίπτωση.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα doctors.

doctor_rank_update_check_trigger: Διασφαλίζει την τήρηση των περιορισμών ιεραρχίας κατά την τροποποίηση των στοιχείων ενός γιατρού. Πρακτικά ελέγχει εάν:

- Η αλλαγή βαθμίδας ή στοιχείων ενός υπάρχοντος γιατρού παραβιάζει τους κανόνες υποχρεωτικής εποπτείας των Ειδικευόμενων ή την αυτονομία των Διευθυντών.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα doctors.

doctor_supervisor_insert_trigger: Αποτρέπει σφάλματα λογικής στη δομή διοίκησης του ιατρικού προσωπικού κατά την εισαγωγή. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα doctors.

doctor_supervisor_update_trigger: Ελέγχει την εγκυρότητα των σχέσεων αναφοράς κατά την αλλαγή ιεραρχίας ενός γιατρού. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα doctors.

check_for_admins_in_procedure_trigger: Περιορίζει τη συμμετοχή σε ιατρικές πράξεις αποκλειστικά στο εξουσιοδοτημένο υγειονομικό προσωπικό. Πρακτικά ελέγχει εάν:

- Ο υπάλληλος που επιχειρείται να προστεθεί ως βοηθός σε μια ιατρική πράξη ανήκει στο διοικητικό προσωπικό (admin_staff), εμποδίζοντας την καταχώρηση και επιτρέποντας μόνο γιατρούς και νοσηλευτές.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα procedure_assistants.

check_admission_rating_trigger: Περιορίζει χρονικά τη δυνατότητα αξιολόγησης των υπηρεσιών του νοσοκομείου από τον ασθενή. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα admission_ratings.

check_doctor_rating_insert_trigger: Επαληθεύει τη σχέση ασθενούς-γιατρού πριν την καταχώρηση μιας νέας βαθμολογίας. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα doctor_ratings.

check_doctor_rating_update_trigger: Διασφαλίζει τη νομιμότητα της σχέσης ασθενούς-γιατρού κατά την τροποποίηση μιας αξιολόγησης. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα doctor_ratings.

check_lab_exam_window_insert_trigger: Διασφαλίζει ότι οι εργαστηριακές εξετάσεις εκτελούνται αυστηρά κατά τη διάρκεια της νοσηλείας. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα lab_exams.

check_lab_exam_window_update_trigger: Ελέγχει τη χρονική εγκυρότητα των εξετάσεων σε σχέση με τη νοσηλεία κατά την αλλαγή στοιχείων. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα lab_exams.

check_procedure_window_insert_trigger: Εγγυάται ότι οι ιατρικές πράξεις πραγματοποιούνται εντός των ημερομηνιών εισαγωγής και εξιτηρίου. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα procedure_executions.

check_procedure_window_update_trigger: Επαναφέρει τον χρονικό έλεγχο συμβατότητας με τη νοσηλεία κατά την τροποποίηση μιας επέμβασης. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα procedure_executions.

check_patient_drug_allergies_trigger: Προστατεύει την υγεία του ασθενούς ελέγχοντας για αλλεργικές αντιδράσεις κατά τη συνταγογράφηση. Πρακτικά ελέγχει εάν:

- Το φάρμακο που πάει να συνταγογραφηθεί (drug_id) περιέχει κάποια δραστική ουσία η οποία είναι καταγεγραμμένη στις αλλεργίες του συγκεκριμένου ασθενούς (patient_amka). Αν βρεθεί ταύτιση, η συνταγή απορρίπτεται άμεσα.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα prescriptions.

procedure_overlap_insert_trigger: Αποτρέπει τον ταυτόχρονο διπλοπρογραμματισμό χειρουργικών αιθουσών και κύριων ιατρών. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα procedure_executions.

procedure_overlap_update_trigger: Διασφαλίζει την απουσία χρονικών συγκρούσεων για αίθουσες και γιατρούς κατά την ενημέρωση στοιχείων μιας επέμβασης. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα procedure_executions.

shift_staffing_insert_trigger: Ελέγχει την τήρηση των μηνιαίων ορίων εργασίας και των κανόνων ασφαλείας κατά τη στελέχωση μιας βάρδιας. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα shift_staffing.

shift_staffing_update_trigger: Επανελέγχει τα μηνιαία όρια και τις συνεχόμενες νύχτες εργασίας κατά την τροποποίηση μιας ανάθεσης. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα shift_staffing.

prevent_consecutive_shift_insert_trigger: Επιβάλλει την υποχρεωτική μεσολάβηση χρόνου ανάπαυσης για το προσωπικό κατά τη νέα ανάθεση. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα shift_staffing.

prevent_consecutive_shift_update_trigger: Διασφαλίζει την τήρηση των κανόνων ανάπαυσης του προσωπικού κατά την αλλαγή στοιχείων βάρδιας. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα shift_staffing.

after_staffing_insert_trigger: Επαναξιολογεί δυναμικά την επάρκεια της βάρδιας αμέσως μετά την προσθήκη ενός εργαζομένου. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER INSERT trigger στον πίνακα shift_staffing.

after_staffing_delete_trigger: Ενημερώνει την κατάσταση στελέχωσης της βάρδιας σε πραγματικό χρόνο μετά από αφαίρεση προσωπικού. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως AFTER DELETE trigger στον πίνακα shift_staffing.

validate_shift_on_lock_trigger: Ελέγχει αν πληρούνται οι ελάχιστες νόμιμες προϋποθέσεις ασφαλούς στελέχωσης πριν το οριστικό κλείσιμο της βάρδιας. Πρακτικά:

- Καλεί το αντίστοιχο procedure (βλ. παράγραφο 1.5).
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE UPDATE trigger στον πίνακα shifts.

protect_locked_shifts_trigger: «Κλειδώνει» τις ολοκληρωμένες βάρδιες αποτρέποντας εκ των υστέρων προσθήκες προσωπικού. Πρακτικά ελέγχει εάν:

- Η βάρδια στην οποία επιχειρείται η προσθήκη νέου υπαλλήλου έχει ήδη κατάσταση 'completed'. Αν ναι, ο trigger μπλοκάρει την ενέργεια για την αποτροπή αλλοίωσης των ιστορικών δεδομένων.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE INSERT trigger στον πίνακα shift_staffing.

protect_locked_shifts_delete_trigger: Προστατεύει το αρχείο ολοκληρωμένων βαρδιών από εκ των υστέρων αφαιρέσεις προσωπικού. Πρακτικά ελέγχει εάν:

- Η βάρδια από την οποία επιχειρείται η διαγραφή ενός υπαλλήλου είναι ήδη κλειδωμένη ως 'completed', απαγορεύοντας τη μεταβολή της ιστορικής στελέχωσης.
- Τον καλεί η βάση δεδομένων αυτόματα ως BEFORE DELETE trigger στον πίνακα shift_staffing.

1.5 Procedures

Στην αρχιτεκτονική της βάσης μας, οι Stored Procedures είναι πίσω από την αυτοματοποιημένη τήρηση των κανόνων του νοσοκομείου. Συγκεκριμένα, χρησιμοποιήσαμε διαδικασίες για να διαχειριστούμε σύνθετη λογική που υπερβαίνει τα απλά constraints, όπως την αναδρομική ιχνηλάτηση της ιεραρχίας των γιατρών για την αποφυγή κυκλικών αναθέσεων, τον εντοπισμό χρονικών επικαλύψεων σε αίθουσες χειρουργείων και τον έλεγχο των αυστηρών ορίων εργασίας του προσωπικού ανά μήνα. Αυτές οι διαδικασίες καλούνται συστηματικά από Triggers, διασφαλίζοντας ότι κάθε εισαγωγή σε πίνακες όπως ο `shift_staffing` ή ο `procedure_executions` περνάνε πρώτα από ένα αυστηρό φίλτρο.

- **check_supervisor_cycle_proc:** Ελέγχει για κυκλικές εξαρτήσεις στην ιεραρχία των γιατρών (π.χ. ο A να μην επιβλέπει τον εαυτό του ή κάποιον που τον επιβλέπει). Πρακτικά ελέγχει εάν:
 - ένας γιατρός προσπαθεί να οριστεί ως επόπτης του εαυτού του.
 - Χρησιμοποιεί αναδρομικό ερώτημα (Recursive CTE) για να εντοπίσει αν ο νέος επόπτης επιβλέπεται ήδη (άμεσα ή έμμεσα) από τον συγκεκριμένο γιατρό, αποτρέποντας τη δημιουργία κύκλου στην ιεραρχία.

Την καλούν τα `triggers`, `doctor_supervisor_insert_trigger` και `doctor_supervisor_update_trigger`, πριν από την εισαγωγή νέου γιατρού ή την ενημέρωση του πεδίου `supervisor_AMKA` στον πίνακα `doctors`.

- **check_if_discharged:** Διασφαλίζει ότι μια ενέργεια (π.χ. αξιολόγηση) γίνεται μόνο αφού ο ασθενής έχει πάρει εξιτήριο. Πρακτικά ελέγχει εάν:
 - ο πεδίο `discharge_date` στην αντίστοιχη νοσηλεία είναι NULL. Αν είναι, σταματάει τη διαδικασία με μήνυμα σφάλματος.

Την καλούν τα `triggers`, `check_admission_rating_trigger`, `check_doctor_rating_insert_trigger` και `check_doctor_rating_update_trigger`, πριν από την προσθήκη ή τροποποίηση βαθμολογίας/αξιολόγησης από ασθενή.

- **check_doctor_treated_patient:** Ελέγχει αν ο γιατρός συμμετείχε πράγματι στη νοσηλεία (μέσω συνταγογράφησης, επέμβασης ή εξετάσεων) πριν επιτραπεί η αξιολόγησή του. Πρακτικά ελέγχει για το αν:
 - ο συγκεκριμένος γιατρός έχει συνταγογραφήσει φάρμακα, έχει εκτελέσει ιατρική πράξη ή έχει ζητήσει εργαστηριακές εξετάσεις για τη συγκεκριμένη νοσηλεία.

Την καλούν τα `triggers`, `check_doctor_rating_insert_trigger` και `check_doctor_rating_update_trigger`, κατά την προσπάθεια βαθμολόγησης ενός συγκεκριμένου γιατρού από έναν ασθενή.

- **check_exam_within_admission:** Επαληθεύει ότι οι εργαστηριακές εξετάσεις γίνονται εντός των ορίων της νοσηλείας. Αυτό γίνεται:
 - συγκρίνοντας την ημερομηνία της εξέτασης με την ημερομηνία εισαγωγής και εξιτηρίου της νοσηλείας.

Την καλούν τα `triggers`, `check_lab_exam_window_insert_trigger` και `check_lab_exam_window_update_trigger`, κατά την καταχώρηση ή αλλαγή ημερομηνίας στο `lab_exams`.

- check_procedure_within_admission:** Επαληθεύει ότι οι ιατρικές επεμβάσεις γίνονται εντός των ορίων της νοσηλείας. Πρακτικά ελέγχει εάν:
 - η έναρξη της επέμβασης είναι εντός του διαστήματος admission_date έως discharge_date.
 Την καλούν τα triggers, check_procedure_window_insert_trigger και check_procedure_window_update_trigger, κατά την εισαγωγή ή ενημέρωση στον πίνακα procedure_executions.
- check_procedure_conflicts:** Ελέγχει αν η αίθουσα ή ο κύριος χειρουργός είναι απασχολημένοι σε άλλη επέμβαση την ίδια ώρα. Πρακτικά ελέγχει εάν:
 - την ίδια χρονική στιγμή (επικάλυψη ωραρίου) η επιλεγμένη αίθουσα ή ο κύριος χειρουργός είναι δεσμευμένοι σε άλλη επέμβαση.
 Την καλούν τα triggers, procedure_overlap_insert_trigger και trg_procedure_overlap_update, πριν από κάθε προγραμματισμό ή αλλαγή ώρας/αίθουσας επέμβασης.
- validate_staff_shift_max_limit:** Ελέγχει το μηνιαίο όριο βαρδιών ανάλογα με την ειδικότητα (Ιατροί: 15, Νοσηλευτές: 20, Διοικητικοί: 25). Πρακτικά μετράει:
 - πόσες βάρδιες έχει κάνει το άτομο τον τρέχοντα μήνα και αν έχουν υπερβεί τα όρια. Τα όρια είναι: Γιατροί (15), Νοσηλευτές (20), Διοικητικοί (25).
 Την καλούν τα triggers, shift_staffing_insert_trigger και shift_staffing_update_trigger, κατά την ανάθεση μιας βάρδιας σε έναν εργαζόμενο.
- validate_consecutive_nights:** Απαγορεύει σε μέλος του προσωπικού να κάνει πάνω από 3 συνεχόμενες νυχτερινές βάρδιες. Πρακτικά ελέγχει για το αν:
 - η ανάθεση νέας νυχτερινής βάρδιας θα δημιουργούσε σειρά πάνω από 3 συνεχόμενες νύχτες εργασίας.
 Την καλούν τα triggers, shift_staffing_insert_trigger και shift_staffing_update_trigger, μόνο όταν η βάρδια που ανατίθεται είναι τύπου 'Night'.
- validate_shift_rest:** Διασφαλίζει τουλάχιστον 8 ώρες ανάπαυσης μεταξύ των βαρδιών του ίδιου ατόμου. Ουσιαστικά υπολογίζει:
 - τη χρονική απόσταση (σε λεπτά) μεταξύ της νέας βάρδιας και οποιασδήποτε άλλης βάρδιας του ίδιου εργαζόμενου. Πρέπει να μεσολαβούν τουλάχιστον 8 ώρες καθαρού χρόνου (που μεταφράζεται σε 16 ώρες διαφορά μεταξύ των ωρών έναρξης, καθώς κάθε βάρδια διαρκεί 8 ώρες).
 Την καλούν τα triggers, prevent_consecutive_shift_insert_trigger και prevent_consecutive_shift_update_trigger, σε κάθε προσπάθεια προσθήκης προσωπικού σε βάρδια.
- validate_shift_requirements:** Ελέγχει αν μια βάρδια έχει το ελάχιστο απαιτούμενο προσωπικό (3 Ιατροί, 6 Νοσηλευτές, 2 Διοικητικοί) και αν υπάρχει επιβλέπων εφόσον υπάρχει ειδικευόμενος. Ουσιαστικά ελέγχει ότι:
 - το ελάχιστο προσωπικό ανά βάρδια είναι: 3 Γιατροί, 6 Νοσηλευτές, 2 Διοικητικοί.
 - αν υπάρχει Ειδικευόμενος (Resident) στη βάρδια, ικανοποιείται η παρουσία τουλάχιστον ενός Επιμελητή Α' ή Διευθυντή.
 Την καλεί το trigger, validate_shift_on_lock_trigger, όταν ο προϊστάμενος προσπαθεί να αλλάξει το status της βάρδιας σε 'Completed'

- **update_shift_staffing_status:** Ενημερώνει αυτόματα το status της βάρδιας σε 'Scheduled' ή 'Understaffed' ανάλογα με τη στελέχωση. Ουσιαστικά ελέγχει:
 - αν τηρούνται τα ελάχιστα νούμερα προσωπικού (3/6/2).
 Την καλούν τα triggers, after_staffing_insert_trigger και after_staffing_delete_trigger, αμέσως μετά από κάθε προσθήκη ή αφαίρεση ατόμου από μια βάρδια.

1.6 Εισαγωγή Δεδομένων

Για να είναι λειτουργική μια βάση πρέπει να έχει δεδομένα. Λόγω της φύσης της εργασίας έχουμε επιλέξει να εισάγουμε δύο κατηγορίες δεδομένων. Πρώτον, πραγματικά δεδομένα, από το ΕΣΥ και δεύτερον συνθετικά δεδομένα της κλινικής μας.

Τα πραγματικά δεδομένα ήταν:

- Κωδικοί ICD-10: Η διεθνής στατιστική ταξινόμηση νόσων και συναφών προβλημάτων υγείας.
- Κλειστά Ενοποιημένα Νοσήλια (KEN): Η επίσημη ελληνική λίστα για την τιμολόγηση των νοσηλείων.
- Ιατρικές Πράξεις και Εργαστηριακές Εξετάσεις: Η επίσημη ελληνική ονοματολογία και κωδικοποίηση ιατρικών πράξεων.
- Φάρμακα και Δραστικές Ουσίες (EMA): Πραγματικά δεδομένα προϊόντων και ουσιών από τον Ευρωπαϊκό Οργανισμό Φαρμάκων (EMA), φιλτραρισμένα για την ελληνική αγορά.

Η επεξεργασία που έγινε σε αυτά τα δεδομένα ήταν η εξής:

1. Φιλτράρισμα Φαρμάκων (EMA)

- Κρατήθηκαν μόνο τα φάρμακα που έχουν άδεια κυκλοφορίας στην Ελλάδα (country == "Greece").
- Αγνοήθηκαν τα δεδομένα φαρμακοεπαγρύπνησης (στήλες 5, 6, 7 του Excel) ως μη απαραίτητα για το project.
- Στα προϊόντα με πολλαπλές δραστικές ουσίες, έγινε split με βάση το χαρακτήρα | ώστε κάθε ουσία να καταγραφεί ξεχωριστά.

2. Κατηγοριοποίηση Πράξεων και Εξετάσεων

Έγινε ένας σημαντικός διαχωρισμός του αρχείου της "Ελληνικής Ονοματολογίας" σε δύο διαφορετικούς πίνακες στη βάση:

- Ιατρικές Πράξεις, θεωρήθηκαν μόνο όσες ανήκουν στις κατηγορίες Α (Αναισθησία) και Β (Χειρουργικές Πράξεις).
- Εργαστηριακές Εξετάσεις, θεωρήθηκαν όσες ανήκουν στις κατηγορίες Γ (Απεικόνιση), Δ (Βιοπαθολογία), Ε (Ιατροδικαστική) και ΣΤ (Ιατρικές Πράξεις - Λοιπά).

3. Καθαρισμός Δεδομένων KEN

- Έγινε μετατροπή του κόστους από την ελληνική μορφή (π.χ. 1.200,50€) σε αριθμητική μορφή συμβατή με SQL (1200.50).
- NULL Values: Αν ένα KEN δεν είχε ορισμένο κόστος, το script το δέχεται ως NULL αντί να το απορρίψει.

4. Γενικός Καθαρισμός Κειμένων

- Σε όλα τα πεδία (ICD-10, KEN κ.λπ.) αφαιρέθηκαν οι χαρακτήρες αλλαγής γραμμής (\n) και τα πολλαπλά κενά, ώστε οι περιγραφές να είναι καθαρές και ευανάγνωστες στη βάση.

Τα συνθετικά δεδομένα ήταν:

Όλα τα δεδομένα που έχουν να κάνουν με τη λειτουργία του νοσοκομείου. Παρήχθησαν μέσω κώδικα Python (`gen_clinical.py`) για λόγους ταχύτητας και ελέγχου των δεδομένων. Κάποια παραδείγματα:

- Προσωπικό: Ονόματα γιατρών, νοσηλευτών και διοικητικών (βασισμένα σε πρότυπα ελληνικών ονομάτων και επωνύμων).
- Ασθενείς: ΑΜΚΑ, ονοματεπώνυμο και στοιχεία επικοινωνίας.
- Κλινικά Γεγονότα: Νοσηλείες (`admissions`), διαλογές (`trriages`), συνταγογραφήσεις, εκτελέσεις ιατρικών πράξεων και εργαστηριακές εξετάσεις.
- Πρόγραμμα Βαρδιών: Οι βάρδιες του προσωπικού ανά τμήμα.
- Αξιολογήσεις: Σχόλια και βαθμολογίες ασθενών για τις υπηρεσίες και τους γιατρούς.

Τα δεδομένα αυτά έχουν παραχθεί με τέτοιο τρόπο ώστε να ικανοποιούν τους εσωτερικούς κανόνες (π.χ. να μην υπάρχουν επικαλύψεις σε χειρουργεία, να τηρούνται τα όρια βαρδιών κ.λπ.) που ορίζονται από τα `triggers` της βάσης.

Φροντίσαμε να υπάρχει:

1. Ρεαλισμός

- Ονοματολογία: Τα ονόματα δεν είναι εντελώς τυχαία. Υπάρχει συνάρτηση (`adjust_greek_surname`) που αλλάζει την κατάληξη των επωνύμων βάσει γένους (π.χ. Παπαδόπουλος -> Παπαδοπούλου).
- Τεχνητά Στοιχεία: Τα ΑΜΚΑ είναι 11ψήφια, τα κινητά ξεκινούν από "69" και τα email ακολουθούν το πρότυπο firstname.lastname@ygeiopolis.gr.

2. Κλινική Λογική και Πιθανότητες

- Triage (Διαλογή): Κάθε επίπεδο επείγοντος (1-5) έχει διαφορετική πιθανότητα νοσηλείας (π.χ. το Επίπεδο 1 έχει 85%, ενώ το 5 μόνο 10%) και διαφορετικούς χρόνους αναμονής.
- Συσχετίσεις: Μια συνταγή ή μια εξέταση δεν ανατίθεται σε τυχαίο γιατρό, αλλά σε κάποιον που είδε τον ασθενή κατά τη διάρκεια της νοσηλείας του, ώστε να μην παραβιάζονται οι περιορισμοί της βάσης.

3. Προσομοίωση Βαρδιών

- Κανείς δεν δουλεύει πάνω από το μηνιαίο όριο (π.χ. 15 βάρδιες για γιατρούς).
- Υπάρχει κενό τουλάχιστον 16 ωρών μεταξύ της έναρξης δύο βαρδιών του ίδιου ατόμου.
- Απαγορεύεται η 3η συνεχόμενη νυχτερινή βάρδια (κανόνας που ελέγχεται και από `database triggers`).

4. Εικόνες και Media

Χρησιμοποιούνται APIs όπως το Unsplash (για φωτογραφίες τμημάτων), το RandomUser.me (για προφίλ γιατρών) και το LoremFlickr (για φάρμακα), ώστε το UI της εφαρμογής να φαίνεται πλήρες και επαγγελματικό.

Αυτά τα script ([generate.py](#)) φτιάχτηκαν με την βοήθεια εργαλείων τεχνητής νοημοσύνης. Εργαλεία όπως το Claude είναι ικανά να παράγουν σωστά συνθετικά δεδομένα που δεν είναι απλώς τυχαία, αλλά έχουν δομή και νόημα. Τα δεδομένα έχουμε φροντίσει να τηρούν το σχήμα της βάσης. Επιπροσθέτως, χρησιμοποιήθηκαν LLMs (Claude και Gemini) σε συμβουλευτικό επίπεδο κατά την

κατασκευή του ER diagram, κατά την υλοποίηση κάποιων queries και προς διευκόλυνση της εγγραφής της αναφοράς.

Πίνακας	Πλήθος Εγγραφών	Λεπτομέρειες
departments	15	Νοσοκομειακά τμήματα
staff (Σύνολο)	870	Γιατροί (300), Νοσηλευτές (450), Διοικητικοί (120)
beds	600	40 κρεβάτια ανά τμήμα
operating_rooms	10	Αίθουσες χειρουργείων
patients	200	Μοναδικοί ασθενείς
admissions	1.000	500 (Base) + 500 (Alexiou)
triages	120	Περιστατικά στα Επείγοντα
prescriptions	300	Ιατρικές συνταγές φαρμάκων
lab_exams	200	Εργαστηριακές εξετάσεις
procedure_executions	600	Χειρουργικές/Ιατρικές πράξεις
shifts	~8.100	3 βάρδιες/μέρα * 15 τμήματα * 180 μέρες
shift_staffing	~89.100	11 άτομα ανά βάρδια (3+6+2)
icd10	~400	Κωδικοί διαγνώσεων
ken	~450	Κωδικοί τιμολόγησης νοσηλείας
ema_products	~200	Φαρμακευτικά σκευάσματα
medical_procedures	~200	Κατάλογος ιατρικών πράξεων
images	~1.000	Φωτογραφίες γιατρών, τμημάτων, φαρμάκων
ratings	~700	Αξιολογήσεις νοσηλείας και γιατρών

1.7 Φόρτωση Δεδομένων: load.sql

Το αρχείο `load.sql` είναι υπεύθυνο για τη φόρτωση των συνθετικών δεδομένων στη βάση `ygeiopolis`. Δεν περιέχει `INSERT` εντολές, αλλά χρησιμοποιεί την εντολή `LOAD DATA LOCAL INFILE` της `MySQL`, η οποία φορτώνει δεδομένα απευθείας από αρχεία `CSV`. Αυτή η προσέγγιση είναι σημαντικά πιο αποδοτική από μαζικά `INSERT`, καθώς παρακάμπτει μεγάλο μέρος της λογικής ανάλυσης γραμμή-προς-γραμμή και μειώνει τη φόρτωση του `MySQL server`.

Τα δεδομένα επιλέγονται από τον φάκελο με τα `CSV`, που παρήγαγε το αρχείο [generate.py](#), με γαυτοματοποιημένο τρόπο. Έχουμε φροντίσει να ξεκαθαρίσουμε το σύνολο χαρακτήρων (`utf8mb4`, για υποστήριξη ελληνικών) καθώς επίσης τον χαρακτήρα που χωρίζει τα κελιά, (εδώ το κόμμα), όπως και το ότι παραλείπεται η πρώτη γραμμή κάθε πίνακα καθώς αποτελεί την κεφαλίδα.

Η σειρά φόρτωσης δεν είναι τυχαία, καθώς ακολουθεί αυστηρά τις εξαρτήσεις ξένων κλειδιών του σχήματος. Φορτώνονται πρώτα ανεξάρτητοι πίνακες (π.χ. `icd10`, `ken_categories`, `active_substances`), ακολουθούν οι πίνακες που εξαρτώνται από αυτούς (π.χ. `ken`, `drugs`, `departments`), και τελευταίοι φορτώνονται οι πίνακες με τις πολυπλοκότερες σχέσεις, όπως νοσηλείες, χορηγήσεις φαρμάκων και χειρουργεία. Για τις `circular` εξαρτήσεις (π.χ. `departments` - `doctors` λόγω του `director_AMKA`) ακολουθεί φόρτωση σε δύο φάσεις: αρχικά με `NULL` διευθυντή, και στη συνέχεια `UPDATE` εντολές αποκαθιστούν τις σχέσεις.

Το `load.sql` παράχθηκε αυτόματα μέσω του script `generate.py`, το οποίο ανέλαβε να αναλύσει το σχήμα της βάσης, να εντοπίσει τις εξαρτήσεις μεταξύ πινάκων, να επιλύσει κυκλικές αναφορές και να παράγει σωστά διατεταγμένες `LOAD DATA` εντολές χωρίς παρέμβαση. Το αποτέλεσμα είναι ένα script έτοιμο προς εκτέλεση, που φορτώνει ολόκληρη τη βάση.

2. Ζητούμενα Queries

Q.01

Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

```
-- Q1: Συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος
SELECT
    d.name AS department_name,
    YEAR(a.admission_date) AS admission_year,
    a.ken_code,
    p.insurance_provider,
    COUNT(a.admission_id) AS total_admissions,
    SUM(a.base_cost) AS total_base_cost,
    SUM(a.extra_cost) AS total_extra_cost,
    SUM(a.base_cost + a.extra_cost) AS total_revenue
FROM admissions a
JOIN departments d ON a.department_id = d.dept_id
JOIN patients p ON a.patient_AMKA = p.AMKA
GROUP BY d.name, YEAR(a.admission_date), a.ken_code, p.insurance_provider
ORDER BY admission_year DESC, d.name, total_revenue DESC;
```

Πρωτον, κάνουμε INNER JOIN τον κεντρικό πίνακα admissions με τους πίνακες departments και patients. Στη συνέχεια, ομαδοποιούμε τις εγγραφές (GROUP BY) χρησιμοποιώντας ένα σύνθετο κλειδί τεσσάρων διαστάσεων: όνομα τμήματος, έτος (κάνοντας extract τη χρονιά μέσω της συνάρτησης YEAR), κωδικό KEN και ασφαλιστικό φορέα. Για κάθε group, εφαρμόζουμε aggregate functions, υπολογίζοντας τον συνολικό όγκο εισαγωγών (COUNT) καθώς και τα αθροίσματα (SUM) του βασικού, του πρόσθετου και του συνολικού κόστους. Τέλος, ταξινομούμε το result set (ORDER BY) φθίνουσα χρονολογικά, αλφαβητικά ανά τμήμα και φθίνουσα βάσει των συνολικών εσόδων.

Q.02

Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτή, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

```
-- Q2: Ιατροί συγκεκριμένης ειδικότητας και επεμβάσεις
SELECT
  s.AMKA,
  s.first_name,
  s.last_name,
  spec.name AS specialty,
  EXISTS (
    SELECT 1 FROM shift_staffing ss
    JOIN shifts sh ON ss.shift_id = sh.shift_id
    WHERE ss.staff_AMKA = s.AMKA AND YEAR(sh.shift_date) = YEAR(CURRENT_DATE)
  ) AS had_shift_this_year,
  (
    SELECT COUNT(*) FROM procedure_executions pe
    WHERE pe.main_doctor_AMKA = s.AMKA
  ) AS surgeries_as_main
FROM doctors doc
JOIN staff s ON doc.AMKA = s.AMKA
JOIN specialties spec ON doc.specialty = spec.code
WHERE spec.name = 'Χειρουργική'
ORDER BY surgeries_as_main DESC;
```

Εκτελούμε INNER JOIN μεταξύ των πινάκων doctors, staff και specialties για την άντληση των δημογραφικών στοιχείων και της ειδικότητας. Στο SELECT clause ενσωματώνουμε δύο correlated subqueries. Το πρώτο αξιοποιεί τον τελεστή EXISTS στους πίνακες των βαρδιών για να ελέγξει αποδοτικά εάν ο ιατρός έχει έστω και μία βάρδια στο τρέχον έτος, επιστρέφοντας μια boolean τιμή. Το δεύτερο subquery εκτελεί COUNT στον πίνακα των επεμβάσεων για να υπολογίσει το πλήθος των χειρουργείων όπου ο εκάστοτε ιατρός συμμετείχε ως κύριος χειρουργός. Τέλος, φιλτράρουμε τις εγγραφές αποκλειστικά για τους χειρουργούς (WHERE spec.name = 'Surgery') και ταξινομούμε το result set σε φθίνουσα σειρά, βάσει του όγκου των διεκπεραιωμένων επεμβάσεων.

Q.03

Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```
-- Q3: Ασθενείς με περισσότερες από 3 νοσηλείες στο ίδιο τμήμα
SELECT
    p.AMKA,
    p.first_name,
    p.last_name,
    d.name AS department_name,
    COUNT(a.admission_id) AS admission_count,
    SUM(a.base_cost + a.extra_cost) AS total_cost
FROM patients p
JOIN admissions a ON p.AMKA = a.patient_AMKA
JOIN departments d ON a.department_id = d.dept_id
GROUP BY p.AMKA, p.first_name, p.last_name, d.name
HAVING COUNT(a.admission_id) > 3
ORDER BY admission_count DESC;
```

Αρχικά, κάνουμε JOIN τους πίνακες των ασθενών, των νοσηλειών και των τμημάτων για να ενώσουμε τα στοιχεία που χρειαζόμαστε. Στη συνέχεια, με την GROUP BY, ομαδοποιούμε τα δεδομένα με βάση τον ασθενή και την κλινική. Έτσι, μπορούμε να χρησιμοποιήσουμε την COUNT για να μετρήσουμε τον αριθμό των εισαγωγών του κάθε ασθενή και τη SUM για να βρούμε το συνολικό κόστος αυτών των νοσηλειών. Η διαλογή γίνεται με την HAVING, με την οποία κρατάμε τελικά μόνο τα άτομα που έχουν περισσότερες από τρεις νοσηλείες. Τέλος, χρησιμοποιούμε την ORDER BY για να ταξινομήσουμε τα αποτελέσματα, φέρνοντας στην κορυφή της λίστας τους ασθενείς με τις περισσότερες εισαγωγές.

Q.04

Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας*.

```
-- Q4: Μέσος όρος αξιολογήσεων για συγκεκριμένο ιατρό
SELECT
    doc.AMKA,
    s.first_name,
    s.last_name,
    ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
    ROUND(AVG(ar.overall), 2) AS avg_overall_admission_rating,
    COUNT(dr.rating_id) AS total_ratings
FROM doctors doc
JOIN staff s ON doc.AMKA = s.AMKA
JOIN doctor_ratings dr ON doc.AMKA = dr.doctor_AMKA
JOIN admissions a ON dr.admission_id = a.admission_id
JOIN admission_ratings ar ON a.admission_id = ar.admission_id
WHERE s.last_name = 'Alexiou' AND s.first_name = 'Michalis'
GROUP BY doc.AMKA, s.first_name, s.last_name;
```

Για αρχή, κάνουμε JOIN αρκετούς πίνακες μαζί: τους γιατρούς, το προσωπικό (για να ελέγξουμε το όνομα), τις αξιολογήσεις των ιατρών, τις εισαγωγές και τις συνολικές αξιολογήσεις της νοσηλείας. Με τη WHERE φιλτράρουμε τα αποτελέσματα ώστε να κρατήσουμε μόνο τα δεδομένα για τον συγκεκριμένο γιατρό. Μετά, χρησιμοποιούμε την GROUP BY για να ομαδοποιήσουμε τις εγγραφές ανά AMKA και επίθετο. Για τους υπολογισμούς, χρησιμοποιούμε την AVG για να βρούμε τον μέσο όρο της ιατρικής φροντίδας και της συνολικής εμπειρίας του ασθενή.

AMKA	first_name	last_name	avg_medical_care_quality	avg_overall_admission_rating	total_ratings
22132041042	Michalis	Alexiou	4.03	3.99	501

* Για τα ερωτήματα Q4 και Q6, να παρουσιάσετε: (α) το αποτέλεσμα EXPLAIN/EXPLAIN ANALYZE, (β) εναλλακτική έκδοση με FORCE INDEX ή hint, (γ) σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων, και (δ) σύντομη ανάλυση (3-5 προτάσεις) των συμπερασμάτων σας. Για να είναι ορατές οι διαφορές στα query plans, θα πρέπει να έχετε ικανό αριθμό δεδομένων.

α)

```
-- (α) EXPLAIN χωρίς index στο staff.last_name

DROP INDEX IF EXISTS idx_staff_last_name ON staff;

EXPLAIN SELECT
  doc.AMKA,
  s.first_name,
  s.last_name,
  ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
  ROUND(AVG(ar.overall), 2) AS avg_overall_admission_rating,
  COUNT(dr.rating_id) AS total_ratings
FROM doctors doc
JOIN staff s ON doc.AMKA = s.AMKA
JOIN doctor_ratings dr ON doc.AMKA = dr.doctor_AMKA
JOIN admissions a ON dr.admission_id = a.admission_id
JOIN admission_ratings ar ON a.admission_id = ar.admission_id
WHERE s.last_name = 'Alexiou' AND s.first_name = 'Michalis'
GROUP BY doc.AMKA, s.first_name, s.last_name;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	doc	index	PRIMARY	PRIMARY	44	NULL	300	Using index
1	SIMPLE	dr	ref	uni_admission_doctor_duplicate,fk_doc_rating_doc	fk_doc_rating_doc	44	ygetopolis.doc.AMKA	1	
1	SIMPLE	ar	eq_ref	PRIMARY	PRIMARY	4	ygetopolis.dr.admission_id	1	
1	SIMPLE	s	eq_ref	PRIMARY	PRIMARY	44	ygetopolis.doc.AMKA	1	Using where
1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	ygetopolis.dr.admission_id	1	Using index

5 rows in set (0.001 sec)

β)

```
-- (β) Εναλλακτική έκδοση με FORCE INDEX στο staff.last_name

CREATE INDEX IF NOT EXISTS idx_staff_last_name ON staff(last_name);

EXPLAIN SELECT
  doc.AMKA,
  s.first_name,
  s.last_name,
  ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
  ROUND(AVG(ar.overall), 2) AS avg_overall_admission_rating,
  COUNT(dr.rating_id) AS total_ratings
FROM doctors doc
JOIN staff s FORCE INDEX (idx_staff_last_name) ON doc.AMKA = s.AMKA
JOIN doctor_ratings dr ON doc.AMKA = dr.doctor_AMKA
JOIN admissions a ON dr.admission_id = a.admission_id
JOIN admission_ratings ar ON a.admission_id = ar.admission_id
WHERE s.last_name = 'Alexiou' AND s.first_name = 'Michalis'
GROUP BY doc.AMKA, s.first_name, s.last_name;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	s	ref	idx_staff_last_name	idx_staff_last_name	202	const	30	Using where; Using index; Using temporary; Using filesort
1	SIMPLE	doc	eq_ref	PRIMARY	PRIMARY	44	ygeiopolis.s.AMKA	1	Using index
1	SIMPLE	dr	ref	uni_admission_doctor_duplicate, fk_doc_rating_doc	fk_doc_rating_doc	44	ygeiopolis.s.AMKA	1	
1	SIMPLE	ar	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.dr.admission_id	1	
1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.dr.admission_id	1	Using index

5 rows in set (0.001 sec)

))

-- (γ) Σύγκριση πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδόσεων

SET profiling = 1;

-- Έκδοση 1: χωρίς index

DROP INDEX IF EXISTS idx_staff_last_name ON staff;

SELECT

```

    doc.AMKA,
    s.first_name,
    s.last_name,
    ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
    ROUND(AVG(ar.overall), 2)              AS avg_overall_admission_rating,
    COUNT(dr.rating_id)                   AS total_ratings

```

FROM doctors doc

JOIN staff s ON doc.AMKA = s.AMKA

JOIN doctor_ratings dr ON doc.AMKA = dr.doctor_AMKA

JOIN admissions a ON dr.admission_id = a.admission_id

JOIN admission_ratings ar ON a.admission_id = ar.admission_id

WHERE s.last_name = 'Alexiou' AND s.first_name = 'Michalis'

GROUP BY doc.AMKA, s.first_name, s.last_name;

-- Έκδοση 2: με FORCE INDEX

CREATE INDEX IF NOT EXISTS idx_staff_last_name ON staff(last_name);

SELECT

```

    doc.AMKA AS doctor_AMKA,
    s.first_name AS doctor_first_name,
    s.last_name AS doctor_last_name,
    ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
    ROUND(AVG(ar.overall), 2)              AS avg_overall_admission_rating,
    COUNT(dr.rating_id)                   AS total_ratings

```

FROM doctors doc

JOIN staff s FORCE INDEX (idx_staff_last_name) ON doc.AMKA = s.AMKA

JOIN doctor_ratings dr ON doc.AMKA = dr.doctor_AMKA

JOIN admissions a ON dr.admission_id = a.admission_id

JOIN admission_ratings ar ON a.admission_id = ar.admission_id

WHERE s.last_name = 'Alexiou' AND s.first_name = 'Michalis'

GROUP BY doc.AMKA, s.first_name, s.last_name;

-- Query_ID 1 = χωρίς index, Query_ID 2 = με FORCE INDEX

SHOW PROFILES;

SET profiling = 0;

```

+-----+-----+-----+
| Query_ID | Duration | Query
+-----+-----+-----+
| 1 | 0.00012450 | DROP INDEX IF EXISTS idx_staff_last_name ON staff
| 2 | 0.00221324 | SELECT
doc.AMKA AS doctor_AMKA,
s.last_name AS doctor_last_name,
ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
ROUND(AVG(ar.overall), 2) AS avg_overall_admission_rating,
COUNT(dr.rating_id)
| 3 | 0.01515256 | CREATE INDEX IF NOT EXISTS idx_staff_last_name ON staff(last_name)
| 4 | 0.00125660 | SELECT
doc.AMKA AS doctor_AMKA,
s.last_name AS doctor_last_name,
ROUND(AVG(dr.medical_care_quality), 2) AS avg_medical_care_quality,
ROUND(AVG(ar.overall), 2) AS avg_overall_admission_rating,
COUNT(dr.rating_id)
+-----+-----+-----+
4 rows in set (0.000 sec)

```

Query_ID	Query	Duration
2	SELECT χωρίς index	2.21 ms
4	SELECT με FORCE INDEX	1.26 ms

δ)

Χωρίς index στο staff.last_name, ο optimizer επιλέγει να ξεκινήσει από τον πίνακα doctors και να σκανάρει και τους 300 γιατρούς μέσω του PRIMARY KEY, φιλτράροντας το staff με Using where σε κάθε επανάληψη. Με τον index idx_staff_last_name, ο optimizer (αναγκαστικά μέσω FORCE INDEX) αλλάζει στρατηγική: ξεκινά από τον staff και ανακτά απευθείας τις ~30 εγγραφές 'Alexiou' με index lookup (type=ref). Παρόλο που αυτή η προσέγγιση εισάγει Using temporary; Using filesort για το GROUP BY, ο συνολικός χρόνος μειώνεται από 2.21ms σε 1.26ms (~1.76x). Σε μεγαλύτερο πίνακα staff, ο optimizer θα επέλεγε αυτόματα τον index χωρίς FORCE INDEX, καθώς το κόστος του full scan θα ξεπερνούσε το overhead του filesort.

Q.05

Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

```
-- Q5: Νέοι ιατροί (<35 ετών) με τις περισσότερες χειρουργικές επεμβάσεις
SELECT
    s.AMKA,
    s.first_name,
    s.last_name,
    s.age,
    COUNT(pe.execution_id) AS total_surgeries
FROM doctors doc
JOIN staff s          ON doc.AMKA          = s.AMKA
JOIN procedure_executions pe ON doc.AMKA      = pe.main_doctor_AMKA
JOIN medical_procedures mp  ON pe.procedure_code = mp.procedure_code
WHERE s.age < 35
      AND mp.category = 'χειρουργική'
GROUP BY s.AMKA, s.first_name, s.last_name, s.age
ORDER BY total_surgeries DESC;
```

Αρχικά, κάνουμε JOIN τους πίνακες των γιατρών, του προσωπικού, των ιατρικών πράξεων και των εκτελέσεών τους. Με τη WHERE φιλτράρουμε τα δεδομένα για να κρατήσουμε μόνο όσους είναι κάτω των 35 και τις πράξεις που ανήκουν στην κατηγορία 'Surgical'. Στη συνέχεια, χρησιμοποιούμε την GROUP BY για να ομαδοποιήσουμε τις εγγραφές ανά γιατρό. Έτσι, με την COUNT μετράμε πόσα χειρουργεία έκανε ο καθένας. Τέλος, βάζουμε την ORDER BY για να φέρουμε στην κορυφή της λίστας αυτούς με τις περισσότερες επεμβάσεις.

Q.06

Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλείων του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

```
-- Q06: Ιστορικό νοσηλείων συγκεκριμένου ασθενή με διαγνώσεις, συνταγογραφήσεις,
κόστος, εξετάσεις και μέσο όρο αξιολόγησης
SELECT
    p.AMKA,
    p.first_name,
    p.last_name,
    a.admission_id,
    DATE_FORMAT(a.admission_date, '%Y-%m-%d') AS admission_date,
    DATE_FORMAT(a.discharge_date, '%Y-%m-%d') AS discharge_date,
    DATEDIFF(
        COALESCE(a.discharge_date, CURDATE()),
        a.admission_date
    ) AS days_stayed,
    d.name AS department,
    CONCAT(i1.icd10_code, ' - ', i1.description) AS admission_diagnosis,
    CONCAT(i2.icd10_code, ' - ', i2.description) AS discharge_diagnosis,
    a.total_cost,
    ROUND((ar.nursing_quality + ar.cleanliness + ar.food + ar.overall) / 4.0, 2)
AS avg_admission_rating,
    GROUP_CONCAT(
        DISTINCT dr.name
        ORDER BY dr.name
        SEPARATOR ' | '
    ) AS prescriptions,
    GROUP_CONCAT(
        DISTINCT et.name
        ORDER BY et.name
        SEPARATOR ' | '
    ) AS lab_exams
FROM patients p
JOIN admissions a ON p.AMKA = a.patient_AMKA
JOIN departments d ON a.department_id = d.dept_id
JOIN icd10 i1 ON a.admission_diagnosis_code = i1.icd10_code
LEFT JOIN icd10 i2 ON a.discharge_diagnosis_code = i2.icd10_code
LEFT JOIN admission_ratings ar ON a.admission_id = ar.admission_id
LEFT JOIN prescriptions pr ON a.admission_id = pr.admission_id
LEFT JOIN drugs dr ON pr.drug_id = dr.drug_id
```

```
LEFT JOIN lab_exams le      ON a.admission_id = le.admission_id
LEFT JOIN exam_types et     ON le.exam_code   = et.exam_code
WHERE p.last_name = 'Papadopoulos'
GROUP BY a.admission_id, p.first_name, p.last_name, p.AMKA,
         a.admission_date, a.discharge_date, a.base_cost, a.extra_cost,
         d.name, i1.icd10_code, i1.description, i2.icd10_code, i2.description,
         ar.nursing_quality, ar.cleanness, ar.food, ar.overall
ORDER BY a.admission_date DESC;
```

Ξεκινάμε από τον πίνακα patients και κάνουμε INNER JOIN με τον admissions για να ανακτήσουμε το πλήρες ιστορικό νοσηλειών του ασθενούς. Συνδέουμε με τον departments για το όνομα του τμήματος και με τον icd10 δύο φορές, μία INNER JOIN για τη διάγνωση εισαγωγής και μία LEFT JOIN για τη διάγνωση εξόδου, η οποία ενδέχεται να μην έχει συμπληρωθεί ακόμη. Το days_stayed υπολογίζεται με DATEDIFF, χρησιμοποιώντας COALESCE με CURDATE() για να καλύψει τις ενεργές νοσηλείες χωρίς discharge_date, ενώ το total_cost αντλείται απευθείας από τη virtual column του admissions σύμφωνα με το μοντέλο KEN. Με LEFT JOIN στον πίνακα admission_ratings υπολογίζουμε τον μέσο όρο των τεσσάρων Likert κριτηρίων (nursing_quality, cleanliness, food, overall) ως avg_admission_rating, ενώ με αντίστοιχα LEFT JOIN στους prescriptions, drugs, lab_exams και exam_types συγκεντρώνουμε φάρμακα και εξετάσεις της κάθε νοσηλείας, τα οποία συνενώνουμε με GROUP_CONCAT(DISTINCT ...) σε ενιαία πεδία για αναγνωσιμότητα. Φιλτράρουμε στον συγκεκριμένο ασθενή με WHERE στο last_name και ταξινομούμε το result set σε φθίνουσα χρονολογική σειρά εισαγωγής.

 $\alpha)$

```
-- (α) EXPLAIN χωρίς index στο patients.last_name

DROP INDEX IF EXISTS idx_patients_last_name ON patients;

EXPLAIN SELECT
  p.AMKA,
  p.first_name,
  p.last_name,
  a.admission_id,
  DATE_FORMAT(a.admission_date, '%Y-%m-%d') AS admission_date,
  DATE_FORMAT(a.discharge_date, '%Y-%m-%d') AS discharge_date,
  DATEDIFF(
    COALESCE(a.discharge_date, CURDATE()),
    a.admission_date
  ) AS days_stayed,
  a.total_cost,
```

```

        ROUND((ar.nursing_quality + ar.cleanliness + ar.food + ar.overall) / 4.0, 2)
AS avg_admission_rating,
    CONCAT(i1.icd10_code, ' - ', i1.description)        AS admission_diagnosis,
    CONCAT(i2.icd10_code, ' - ', i2.description)        AS discharge_diagnosis,
    a.base_cost + a.extra_cost                          AS total_cost,
    GROUP_CONCAT(
        DISTINCT dr.name
        ORDER BY dr.name
        SEPARATOR ' | '
    )
    )                                                    AS prescriptions,
    GROUP_CONCAT(
        DISTINCT et.name
        ORDER BY et.name
        SEPARATOR ' | '
    )
    )                                                    AS lab_exams
FROM patients p
JOIN admissions a      ON p.AMKA              = a.patient_AMKA
JOIN departments d     ON a.department_id     = d.dept_id
JOIN icd10 i1          ON a.admission_diagnosis_code = i1.icd10_code
LEFT JOIN icd10 i2     ON a.discharge_diagnosis_code = i2.icd10_code
LEFT JOIN admission_ratings ar ON a.admission_id = ar.admission_id
LEFT JOIN prescriptions pr ON a.admission_id = pr.admission_id
LEFT JOIN drugs dr      ON pr.drug_id        = dr.drug_id
LEFT JOIN lab_exams le  ON a.admission_id = le.admission_id
LEFT JOIN exam_types et ON le.exam_code      = et.exam_code
WHERE p.last_name = 'Papadopoulos'
GROUP BY a.admission_id, p.first_name, p.last_name, p.AMKA,
        a.admission_date, a.discharge_date, a.base_cost, a.extra_cost,
        d.name, i1.icd10_code, i1.description, i2.icd10_code, i2.description,
        ar.nursing_quality, ar.cleanliness, ar.food, ar.overall
ORDER BY a.admission_date DESC;

```

id	select type	table	type	possible keys	key	key_len	ref	rows	Extra
1	SIMPLE	d	ALL	PRIMARY	<null>	<null>	<null>	15	Using temporary; Using <u>filesort</u>
1	SIMPLE	a	ref	<u>fk_adm_patient</u> , <u>fk_adm_dept</u> , <u>fk_adm_diag_in</u>	<u>fk_adm_dept</u>	4	<u>ygeiopolis.d.dept_id</u>	1	
1	SIMPLE	p	<u>eq_ref</u>	PRIMARY	PRIMARY	44	<u>ygeiopolis.a.patient_amka</u>	1	Using where
1	SIMPLE	i1	<u>eq_ref</u>	PRIMARY	PRIMARY	30	<u>ygeiopolis.a.admission_diagnosis_code</u>	1	
1	SIMPLE	i2	<u>eq_ref</u>	PRIMARY	PRIMARY	30	<u>ygeiopolis.a.discharge_diagnosis_code</u>	1	Using where
1	SIMPLE	ar	<u>eq_ref</u>	PRIMARY	PRIMARY	4	<u>ygeiopolis.a.admission_id</u>	1	
1	SIMPLE	pr	ref	<u>fk_presc_admi_id</u>	<u>fk_presc_admi_id</u>	4	<u>ygeiopolis.a.admission_id</u>	1	
1	SIMPLE	dr	<u>eq_ref</u>	PRIMARY	PRIMARY	4	<u>ygeiopolis.pr.drug_id</u>	1	Using where
1	SIMPLE (flat, BNL join)	le	ALL	<u>fk_exam_admission</u>	<null>	<null>	<null>	700	Using where; Using join buffer
1	SIMPLE	et	<u>eq_ref</u>	PRIMARY	PRIMARY	4	<u>ygeiopolis.le.exam_code</u>	1	Using where

β)

```
-- (β) Εναλλακτική έκδοση με FORCE INDEX στο patients.last_name

CREATE INDEX IF NOT EXISTS idx_patients_last_name ON patients(last_name);

EXPLAIN SELECT
  p.AMKA,
  p.first_name,
  p.last_name,
  a.admission_id,
  DATE_FORMAT(a.admission_date, '%Y-%m-%d') AS admission_date,
  DATE_FORMAT(a.discharge_date, '%Y-%m-%d') AS discharge_date,
  DATEDIFF(
    COALESCE(a.discharge_date, CURDATE()),
    a.admission_date
  ) AS days_stayed,
  d.name AS department,
  CONCAT(i1.icd10_code, ' - ', i1.description) AS
admission_diagnosis,
```

```

        CONCAT(i2.icd10_code, ' - ', i2.description)        AS
discharge_diagnosis,
        a.base_cost + a.extra_cost                        AS total_cost,
        GROUP_CONCAT(
            DISTINCT dr.name
            ORDER BY dr.name
            SEPARATOR ' | '
        )
        )
        AS prescriptions,
        GROUP_CONCAT(
            DISTINCT et.name
            ORDER BY et.name
            SEPARATOR ' | '
        )
        )
        AS lab_exams
FROM patients p FORCE INDEX (idx_patients_last_name)
JOIN admissions a      ON p.AMKA          = a.patient_AMKA
JOIN departments d     ON a.department_id = d.dept_id
JOIN icd10 i1          ON a.admission_diagnosis_code = i1.icd10_code
LEFT JOIN icd10 i2     ON a.discharge_diagnosis_code = i2.icd10_code
LEFT JOIN prescriptions pr ON a.admission_id = pr.admission_id
LEFT JOIN drugs dr      ON pr.drug_id     = dr.drug_id
LEFT JOIN lab_exams le   ON a.admission_id = le.admission_id
LEFT JOIN exam_types et  ON le.exam_code  = et.exam_code
WHERE p.last_name = 'Papadopoulos'
GROUP BY a.admission_id, p.first_name, p.last_name, p.AMKA,
        a.admission_date, a.discharge_date, a.base_cost, a.extra_cost,
        d.name, i1.icd10_code, i1.description, i2.icd10_code, i2.description
ORDER BY a.admission_date DESC;

```

id	select type	table	type	possible keys	key	key len	ref	rows	Extra
1	SIMPLE	p	ref	idx_patients_last_name	idx_patients_last_name	202	const	7	Using index condition;
1	SIMPLE	a	ref	fk_adm_patient, fk_adm_dept, fk_adm_diag_in	fk_adm_patient	44	ygeiopolis.p.amka	1	Using temporary; Using filesort
1	SIMPLE	d	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.a.department_id	1	
1	SIMPLE	i1	eq_ref	PRIMARY	PRIMARY	30	ygeiopolis.a.admission_diagnosis_code	1	
1	SIMPLE	i2	eq_ref	PRIMARY	PRIMARY	30	ygeiopolis.a.discharge_diagnosis_code	1	Using where
1	SIMPLE	ar	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.a.admission_id	1	
1	SIMPLE	pr	ref	fk_presc_admi_id	fk_presc_admi_id	4	ygeiopolis.a.admission_id	1	
1	SIMPLE	dr	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.pr.drug_id	1	Using where
1	SIMPLE	le	ALL	fk_exam_admission	<null>	<null>	<null>	700	Using where; Using join buffer (flat, BNL join)
1	SIMPLE	et	eq_ref	PRIMARY	PRIMARY	4	ygeiopolis.le.exam_code	1	Using where

γ)

```

SET profiling = 1;

-- Έκδοση 1: χωρίς index
DROP INDEX IF EXISTS idx_patients_last_name ON patients;

SELECT
    p.AMKA,
    p.first_name,
    p.last_name,
    a.admission_id,
    DATE_FORMAT(a.admission_date, '%Y-%m-%d') AS admission_date,
    DATE_FORMAT(a.discharge_date, '%Y-%m-%d') AS discharge_date,
    DATEDIFF(
        COALESCE(a.discharge_date, CURDATE()),
        a.admission_date
    ) AS days_stayed,

```

```

        d.name                                AS department,
        CONCAT(i1.icd10_code, ' - ', i1.description) AS admission_diagnosis,
        CONCAT(i2.icd10_code, ' - ', i2.description) AS discharge_diagnosis,
        a.total_cost,
        ROUND((ar.nursing_quality + ar.cleanliness + ar.food + ar.overall) / 4.0, 2)
AS avg_admission_rating,
        GROUP_CONCAT(
            DISTINCT dr.name
            ORDER BY dr.name
            SEPARATOR ' | '
        )
                                                AS prescriptions,
        GROUP_CONCAT(
            DISTINCT et.name
            ORDER BY et.name
            SEPARATOR ' | '
        )
                                                AS lab_exams
FROM patients p
JOIN admissions a      ON p.AMKA          = a.patient_AMKA
JOIN departments d    ON a.department_id = d.dept_id
JOIN icd10 i1         ON a.admission_diagnosis_code = i1.icd10_code
LEFT JOIN icd10 i2    ON a.discharge_diagnosis_code = i2.icd10_code
LEFT JOIN admission_ratings ar ON a.admission_id = ar.admission_id
LEFT JOIN prescriptions pr ON a.admission_id = pr.admission_id
LEFT JOIN drugs dr     ON pr.drug_id      = dr.drug_id
LEFT JOIN lab_exams le  ON a.admission_id = le.admission_id
LEFT JOIN exam_types et ON le.exam_code   = et.exam_code
WHERE p.last_name = 'Papadopoulos'
GROUP BY a.admission_id, p.first_name, p.last_name, p.AMKA,
        a.admission_date, a.discharge_date, a.base_cost, a.extra_cost,
        d.name, i1.icd10_code, i1.description, i2.icd10_code, i2.description,
        ar.nursing_quality, ar.cleanliness, ar.food, ar.overall
ORDER BY a.admission_date DESC;

-- Έκδοση 2: με FORCE INDEX
CREATE INDEX IF NOT EXISTS idx_patients_last_name ON patients(last_name);

SELECT
    p.AMKA,
    p.first_name,
    p.last_name,
    a.admission_id,

```

```

DATE_FORMAT(a.admission_date, '%Y-%m-%d')      AS admission_date,
DATE_FORMAT(a.discharge_date, '%Y-%m-%d')      AS discharge_date,
DATEDIFF(
    COALESCE(a.discharge_date, CURDATE()),
    a.admission_date
)
AS days_stayed,
d.name
AS department,
CONCAT(i1.icd10_code, ' - ', i1.description)    AS admission_diagnosis,
CONCAT(i2.icd10_code, ' - ', i2.description)    AS discharge_diagnosis,
a.total_cost,
ROUND((ar.nursing_quality + ar.cleanliness + ar.food + ar.overall) / 4.0, 2)
AS avg_admission_rating,
GROUP_CONCAT(
    DISTINCT dr.name
    ORDER BY dr.name
    SEPARATOR ' | '
)
AS prescriptions,
GROUP_CONCAT(
    DISTINCT et.name
    ORDER BY et.name
    SEPARATOR ' | '
)
AS lab_exams
FROM patients p FORCE INDEX (idx_patients_last_name)
JOIN admissions a    ON p.AMKA          = a.patient_AMKA
JOIN departments d    ON a.department_id = d.dept_id
JOIN icd10 i1         ON a.admission_diagnosis_code = i1.icd10_code
LEFT JOIN icd10 i2    ON a.discharge_diagnosis_code = i2.icd10_code
LEFT JOIN admission_ratings ar ON a.admission_id = ar.admission_id
LEFT JOIN prescriptions pr ON a.admission_id = pr.admission_id
LEFT JOIN drugs dr      ON pr.drug_id     = dr.drug_id
LEFT JOIN lab_exams le   ON a.admission_id = le.admission_id
LEFT JOIN exam_types et  ON le.exam_code  = et.exam_code
WHERE p.last_name = 'Papadopoulos'
GROUP BY a.admission_id, p.first_name, p.last_name, p.AMKA,
        a.admission_date, a.discharge_date, a.base_cost, a.extra_cost,
        d.name, i1.icd10_code, i1.description, i2.icd10_code, i2.description,
        ar.nursing_quality, ar.cleanliness, ar.food, ar.overall
ORDER BY a.admission_date DESC;

-- Query_ID 1 = χωρίς index, Query_ID 2 = με FORCE INDEX
SHOW PROFILES;

```


SET profiling = 0;

Query_ID	Duration	Query
1	0.01532406	-- Έκδοση 1: χωρίς index DROP INDEX IF EXISTS <u>idx_patients_last_name</u> ON <u>patients</u>
2	0.00420952	SELECT <u>p.AMKA</u> , <u>p.first_name</u> , <u>p.last_name</u> , <u>a.admission_id</u> , DATE_FORMAT(<u>a.admission_date</u> , '%Y-%m-%d') AS <u>admission_date</u> , DATE_FORMAT(<u>a.discharge_date</u> , '%Y-%m-%d') AS <u>discharge_date</u> , DATEDIFF(COALESCE(<u>a.discharge_date</u> , CURDATE()), <u>a.admis</u>
3	0.03478919	-- Έκδοση 2: με FORCE INDEX CREATE INDEX IF NOT EXISTS <u>idx_patients_last_name</u> ON <u>patients</u> (<u>last_name</u>)
4	0.01034868	SELECT <u>p.AMKA</u> , <u>p.first_name</u> , <u>p.last_name</u> , <u>a.admission_id</u> , DATE_FORMAT(<u>a.admission_date</u> , '%Y-%m-%d') AS <u>admission_date</u> , DATE_FORMAT(<u>a.discharge_date</u> , '%Y-%m-%d') AS <u>discharge_date</u> , DATEDIFF(COALESCE(<u>a.discharge_date</u> , CURDATE()), <u>a.admis</u>
5	0.03200628	-- Έκδοση 1: χωρίς index DROP INDEX IF EXISTS <u>idx_patients_last_name</u> ON <u>patients</u>
6	0.00480941	SELECT <u>p.AMKA</u> , <u>p.first_name</u> , <u>p.last_name</u> , <u>a.admission_id</u> , DATE_FORMAT(<u>a.admission_date</u> , '%Y-%m-%d') AS <u>admission_date</u> , DATE_FORMAT(<u>a.discharge_date</u> , '%Y-%m-%d') AS <u>discharge_date</u> , DATEDIFF(COALESCE(<u>a.discharge_date</u> , CURDATE()), <u>a.admis</u>
7	0.0202304	-- Έκδοση 2: με FORCE INDEX CREATE INDEX IF NOT EXISTS <u>idx_patients_last_name</u> ON <u>patients</u> (<u>last_name</u>)
8	0.00320397	SELECT <u>p.AMKA</u> , <u>p.first_name</u> , <u>p.last_name</u> , <u>a.admission_id</u> , DATE_FORMAT(<u>a.admission_date</u> , '%Y-%m-%d') AS <u>admission_date</u> , DATE_FORMAT(<u>a.discharge_date</u> , '%Y-%m-%d') AS <u>discharge_date</u> , DATEDIFF(COALESCE(<u>a.discharge_date</u> , CURDATE()), <u>a.admis</u>

δ)

Στην έκδοση χωρίς index ο optimizer ξεκινά από τον departments με full table scan και κάνει join μέσω των primary keys, με Using temporary και filesort για το ORDER BY. Με FORCE INDEX, η σειρά αναστρέφεται και ξεκινά απευθείας από τον patients μέσω του idx_patients_last_name (type = ref, περίπου 7 rows). Παρά τη βελτίωση στο plan, οι πραγματικοί χρόνοι (SHOW PROFILES) είναι συγκρίσιμοι (3–10 ms), με την FORCE INDEX έκδοση να μην αποδίδει σταθερά γρηγορότερα. Αυτό οφείλεται στο μικρό μέγεθος του patients (200 εγγραφές), όπου το κέρδος του index lookup έναντι του full scan είναι αμελητέο, ενώ ο πραγματικός bottleneck παραμένει το BNL join με τον lab_exams (700 rows) που δεν επηρεάζεται. Συμπερασματικά, τα index hints αποδίδουν μόνο όταν ο όγκος δεδομένων το δικαιολογεί και όταν ο indexed πίνακας είναι όντως το bottleneck.

Q.07

Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

```
SELECT
    sub.name                                     AS substance,
    COUNT(DISTINCT pa.patient_AMKA)            AS allergic_patients,
    COUNT(DISTINCT dcs.drug_id)                 AS drugs_containing
FROM active_substances sub
LEFT JOIN patient_allergies pa                 ON sub.substance_id = pa.substance_id
LEFT JOIN drug_contains_substances dcs        ON sub.substance_id = dcs.substance_id
GROUP BY sub.substance_id, sub.name
HAVING allergic_patients > 0 OR drugs_containing > 0
ORDER BY allergic_patients DESC;
```

Εκτελούμε LEFT JOIN της οντότητας active_substances με τους πίνακες patient_allergies και drug_contains_substances, ώστε να συμπεριλαμβάνονται όλες οι δραστικές ουσίες ακόμη και αν δεν εμφανίζονται σε καμία από τις δύο σχέσεις. Στο SELECT clause χρησιμοποιούμε COUNT(DISTINCT ...) για κάθε σχέση, αποτρέποντας διπλομέτρηση που θα προέκυπτε από το καρτεσιανό γινόμενο των δύο joins. Ομαδοποιούμε ανά δραστική ουσία και με το HAVING φιλτράρουμε εκείνες που εμφανίζονται είτε σε αλλεργία είτε σε φάρμακο, αποκλείοντας ουσίες χωρίς καμία σύνδεση. Τέλος, ταξινομούμε το result set σε φθίνουσα σειρά βάσει του πλήθους αλλεργικών ασθενών.

Q.08

Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

```
SET @target_date = '2026-01-05';
SET @target_dept = 1;

SELECT
    s.AMKA,
    s.first_name,
    s.last_name,
    s.staff_type,
    COALESCE(doc.rank, n.rank, a.position) AS role,
    COALESCE(dd.dept_name, dep_n.name, dep_a.name) AS department
FROM staff s
LEFT JOIN doctors doc          ON s.AMKA = doc.AMKA
LEFT JOIN (
    SELECT dd.doctor_AMKA, GROUP_CONCAT(dep.name SEPARATOR ', ') AS dept_name
    FROM doctor_departments dd
    JOIN departments dep ON dd.dept_id = dep.dept_id
    GROUP BY dd.doctor_AMKA
) dd                          ON doc.AMKA = dd.doctor_AMKA
LEFT JOIN nurses n            ON s.AMKA = n.AMKA
LEFT JOIN departments dep_n   ON n.dept_id = dep_n.dept_id
LEFT JOIN admin_staff a       ON s.AMKA = a.AMKA
LEFT JOIN departments dep_a   ON a.dept_id = dep_a.dept_id
WHERE (
    -- Ανήκει στο συγκεκριμένο τμήμα
    EXISTS (SELECT 1 FROM doctor_departments dd2 WHERE dd2.doctor_AMKA = s.AMKA AND
dd2.dept_id = @target_dept)
    OR n.dept_id = @target_dept
    OR a.dept_id = @target_dept
)
AND NOT EXISTS (
    SELECT NULL
    FROM shift_staffing ss
    JOIN shifts sh ON ss.shift_id = sh.shift_id
    WHERE ss.staff_AMKA = s.AMKA
        AND sh.shift_date = @target_date
        AND sh.dept_id    = @target_dept
)
ORDER BY s.staff_type, s.last_name;
```

Ξεκινάμε από τον πίνακα staff και κάνουμε LEFT JOIN με τους τρεις υποτύπους (doctors, nurses, admin_staff) ώστε να ανακτήσουμε ρόλο και τμήμα ανεξάρτητα από την κατηγορία προσωπικού. Για τους ιατρούς, που ανήκουν σε πολλά τμήματα (M:N), χρησιμοποιούμε subquery με GROUP_CONCAT πάνω στον doctor_departments για να εμφανίσουμε όλα τα τμήματά τους συνενωμένα. Με τη συνάρτηση COALESCE επιλέγουμε δυναμικά τη μη-NULL τιμή από την κατάλληλη υποκατηγορία για τις στήλες role και department. Στο WHERE clause κρατάμε μόνο μέλη που ανήκουν στο τμήμα-στόχο (μέσω EXISTS για ιατρούς ή απευθείας FK σύγκριση για

νοσηλευτές και διοικητικούς) και ταυτόχρονα εφαρμόζουμε NOT EXISTS subquery στον shift_staffing για να αποκλείσουμε όσους έχουν ήδη προγραμματισμένη εφημερία στη συγκεκριμένη ημερομηνία και τμήμα. Τέλος, ταξινομούμε το result set ανά κατηγορία προσωπικού και επώνυμο.

Q.09

Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```
WITH patient_days AS (  
    SELECT  
        p.AMKA,  
        CONCAT(p.first_name, ' ', p.last_name) AS patient_name,  
        SUM(DATEDIFF(a.discharge_date, a.admission_date)) AS total_days  
    FROM patients p  
    JOIN admissions a ON p.AMKA = a.patient_AMKA  
    WHERE YEAR(a.admission_date) = 2026  
        AND a.discharge_date IS NOT NULL  
    GROUP BY p.AMKA, p.first_name, p.last_name  
    HAVING total_days > 15  
)  
SELECT  
    a.patient_name AS patient_1,  
    b.patient_name AS patient_2,  
    a.total_days  
FROM patient_days a  
JOIN patient_days b ON a.total_days = b.total_days  
                    AND a.AMKA < b.AMKA  
ORDER BY a.total_days DESC, a.patient_name;
```

Ξεκινάμε με Common Table Expression (CTE) που υπολογίζει για κάθε ασθενή το συνολικό άθροισμα ημερών νοσηλείας εντός του 2026 μέσω `SUM(DATEDIFF(...))`, φιλτράροντας μόνο τις ολοκληρωμένες νοσηλείες (`discharge_date IS NOT NULL`) και κρατώντας με `HAVING` όσους ξεπερνούν τις 15 ημέρες. Στο κύριο query κάνουμε self-join του CTE με τον εαυτό του πάνω στη συνθήκη `total_days = total_days` για να εντοπίσουμε ζεύγη ασθενών με ίδιο πλήθος ημερών. Η συνθήκη `a.AMKA < b.AMKA` αποτρέπει διπλότυπα ζεύγη και αυτο-ταιριάσματα (π.χ. ο ασθενής A με τον εαυτό του, ή τα συμμετρικά A-B και B-A). Τέλος, ταξινομούμε το result set σε φθίνουσα σειρά βάσει διάρκειας νοσηλείας και αλφαβητικά κατά όνομα ασθενούς.

Q.10

Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```
WITH prescription_substances AS (  
    SELECT DISTINCT  
        p.admission_id,  
        p.patient_AMKA,  
        a.substance_id,  
        a.name as sub_name  
    FROM prescriptions p  
    JOIN drug_contains_substances d ON p.drug_id = d.drug_id  
    JOIN active_substances a ON d.substance_id = a.substance_id  
)  
SELECT  
    p1.sub_name AS substance_1,  
    p2.sub_name AS substance_2,  
    COUNT(*) AS frequency  
FROM prescription_substances p1  
JOIN prescription_substances p2 ON p1.admission_id = p2.admission_id  
    AND p1.patient_AMKA = p2.patient_AMKA  
    AND p1.substance_id < p2.substance_id  
GROUP BY substance_1, substance_2  
ORDER BY frequency DESC  
LIMIT 3;
```

Ορίζουμε αρχικά ένα κοινό CTE με το όνομα `prescription_substances`, η οποία πραγματοποιεί INNER JOIN μεταξύ των πινάκων `prescriptions`, `drug_contains_substances` και `active_substances`. Με τη χρήση του χαρακτηρισμού `DISTINCT`, απομονώνουμε τις μοναδικές δραστικές ουσίες ανά νοσηλεία και ασθενή, προλαμβάνοντας τεχνητές διπλοεγγραφές από επαναλαμβανόμενες συνταγές του ίδιου φαρμάκου. Στο κύριο ερώτημα, εκτελούμε self-join του CTE με τον εαυτό της, συσχετίζοντας τις εγγραφές που μοιράζονται το ίδιο `admission_id` και `patient_AMKA`. Εφαρμόζοντας το φιλτράρισμα `p1.substance_id < p2.substance_id`, διασφαλίζουμε ότι κάθε ζεύγος ουσιών θα εμφανιστεί μόνο μία φορά (αποκλείοντας συνδυασμούς της ίδιας ουσίας ή αντιμετάθεση της σειράς των υπάρχοντων σχέσεων). Τέλος, ομαδοποιούμε τα αποτελέσματα με βάση τα ονόματα των δύο ουσιών, καταμετρούμε τη συχνότητα συνύπαρξής τους με την `COUNT(*)`, ταξινομούμε σε φθίνουσα σειρά και περιορίζουμε το αποτέλεσμα μέσω της `LIMIT 3` για την ανάδειξη των τριών top συνδυασμών.

Q.11

Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

```
WITH doctor_totals AS (  
    SELECT  
        main_doctor_AMKA,  
        COUNT(*) AS total_procedures  
    FROM procedure_executions  
    WHERE YEAR(start_time) = YEAR(CURDATE())  
    GROUP BY main_doctor_AMKA  
)  
SELECT  
    d.AMKA,  
    s.first_name,  
    s.last_name,  
    dt.total_procedures  
FROM doctor_totals dt  
JOIN doctors d ON dt.main_doctor_AMKA = d.AMKA  
JOIN staff s ON d.AMKA = s.AMKA  
WHERE dt.total_procedures <= (SELECT MAX(total_procedures) FROM doctor_totals) - 5  
ORDER BY dt.total_procedures DESC;
```

Ορίζουμε αρχικά μια κοινή επιτραπέζια έκφραση (CTE) με το όνομα `doctor_totals`, η οποία φιλτράρει τις επεμβάσεις του τρέχοντος έτους (`YEAR(start_time) = YEAR(CURDATE())`) και ομαδοποιεί τα δεδομένα ανά κύριο ιατρό, υπολογίζοντας το συνολικό πλήθος των χειρουργείων του με την εντολή `COUNT(*)`. Έπειτα, συνδέουμε με `INNER JOIN` το CTE με τους πίνακες `doctors` και `staff` βάσει της `table-per-subtype` δομής της ιεραρχίας του προσωπικού, ώστε να αντλήσουμε τα ονοματεπώνυμα των ιατρών. Στο `WHERE`, χρησιμοποιούμε ένα μη συσχετισμένο υποερώτημα για να εντοπίσουμε τον μέγιστο αριθμό επεμβάσεων που πραγματοποιήθηκαν από έναν μόνο ιατρό (`SELECT MAX(total_procedures)`) και φιλτράρουμε τις εγγραφές κρατώντας μόνο όσους έχουν συγκεντρώσει αριθμό μικρότερο ή ίσο από το όριο αυτό μειωμένο κατά 5 μονάδες. Τέλος, ταξινομούμε το `result set` σε φθίνουσα σειρά με βάση τη δραστηριότητά τους, προκειμένου να εμφανίζονται ιεραρχικά οι ιατροί που πλησιάζουν περισσότερο το όριο των 5 μονάδων.

Q.12

Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```
SET @target_week = '2026-05-11';
SELECT
    d.name AS department_name,
    ws.shift_date AS date,
    ws.shift_slot AS shift,
    CASE
        WHEN doc.AMKA IS NOT NULL THEN 'Ιατρός'
        WHEN n.AMKA IS NOT NULL THEN 'Νοσηλευτής'
        WHEN adm.AMKA IS NOT NULL THEN 'Διοικητικός'
    END AS staff_type,
    COALESCE(doc.specialty, n.rank, adm.position) AS role,
    COUNT(ss.staff_AMKA) AS staff_count
FROM shifts ws
JOIN departments d ON ws.dept_id = d.dept_id
JOIN shift_staffing ss ON ws.shift_id = ss.shift_id
LEFT JOIN doctors doc ON ss.staff_AMKA = doc.AMKA
LEFT JOIN nurses n ON ss.staff_AMKA = n.AMKA
LEFT JOIN admin_staff adm ON ss.staff_AMKA = adm.AMKA
WHERE ws.shift_date BETWEEN @target_week AND DATE_ADD(@target_week, INTERVAL 6 DAY)
GROUP BY d.name, ws.shift_date, ws.shift_slot, staff_type, role
ORDER BY ws.shift_date, d.name, ws.shift_slot;
```

Συνδέουμε με INNER JOIN τους πίνακες shifts, departments και shift_staffing για να αποκτήσουμε πρόσβαση στο πρόγραμμα των βαρδιών ανά τμήμα, φιλτράροντας τα δεδομένα για ένα εύρος επτά ημερών από την ορισμένη μεταβλητή μέσω της DATE_ADD. Προκειμένου να αναλύσουμε το προσωπικό στις υποκλάσεις του, εκτελούμε LEFT JOIN με τους πίνακες doctors, nurses και admin_staff βάσει της table-per-subtype δομής της ιεραρχίας (ISA). Στο SELECT, χρησιμοποιούμε μια δομή CASE για να προσδιορίσουμε δυναμικά τον γενικό τύπο του υπαλλήλου αναγνωρίζοντας σε ποιον υποπίνακα εντοπίζεται το AMKA του, ενώ με τη συνάρτηση COALESCE συγχωνεύουμε τις στήλες της ειδικότητας, της βαθμίδας και του ρόλου σε μια κοινή στήλη. Τέλος, ομαδοποιούμε τα αποτελέσματα ανά τμήμα, ημερομηνία, βάρδια, τύπο και ρόλο προσωπικού, καταμετρούμε το πλήθος των εργαζομένων με την COUNT και ταξινομούμε χρονολογικά και ανά τμήμα για την ορθή αποτύπωση των δεδομένων.

Q.13

Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

```
WITH RECURSIVE supervisor_hierarchy AS (  
    SELECT  
        d.amka AS leaf_amka,  
        d.supervisor_amka AS next_supervisor_amka,  
        CAST(CONCAT(s_leaf.first_name, ' ', s_leaf.last_name) AS CHAR(100)) AS  
doctor_name,  
        CASE WHEN d.supervisor_amka IS NULL THEN 1 ELSE 2 END AS level,  
        CAST(CASE WHEN d.supervisor_amka IS NULL THEN '/' ELSE CONCAT('/',  
s_sup.first_name, '-', s_sup.last_name, '/') END AS CHAR(500)) AS hierarchy_path  
    FROM doctors d  
    JOIN staff s_leaf ON d.amka = s_leaf.amka  
    LEFT JOIN staff s_sup ON d.supervisor_amka = s_sup.amka  
    UNION ALL  
    SELECT  
        sh.leaf_amka,  
        d.supervisor_amka,  
        sh.doctor_name,  
        sh.level + 1,  
        CONCAT('/', s.first_name, '-', s.last_name, sh.hierarchy_path)  
    FROM supervisor_hierarchy sh  
    INNER JOIN doctors d ON sh.next_supervisor_amka = d.amka  
    INNER JOIN staff s ON d.supervisor_amka = s.amka  
    WHERE d.supervisor_amka IS NOT NULL  
)  
SELECT  
    doctor_name,  
    level,  
    hierarchy_path  
FROM (  
    SELECT *, ROW_NUMBER() OVER(PARTITION BY leaf_amka ORDER BY level DESC) as  
`number`  
    FROM supervisor_hierarchy  
) ranked_hierarchy  
WHERE `number` = 1  
ORDER BY hierarchy_path, doctor_name;
```

Ορίζουμε ένα CTE, το οποίο επιλέγει όλους τους ιατρούς από τον πίνακα doctors και συνδέεται με τον πίνακα staff για να αντλήσει τα στοιχεία τόσο των ίδιων όσο και των άμεσων εποπτών τους, θέτοντας τις αρχικές τιμές για το βάθος και το path της ιεραρχίας. Στο αναδρομικό σκέλος, η έκφραση πραγματοποιεί διαδοχικούς εσωτερικούς συνδέσμους με τον εαυτό της, διατρέχοντας προς τα πάνω τα επίπεδα διοίκησης μέσω του κλειδιού supervisor_amka μέχρι να συναντήσει τον ανώτατο Διευθυντή (όπου η τιμή γίνεται NULL), αυξάνοντας παράλληλα δυναμικά το επίπεδο βαθμίδας και χτίζοντας το συνολικό μονοπάτι ονομάτων. Στο κύριο ερώτημα, εφαρμόζουμε την αναλυτική συνάρτηση παραθύρου ROW_NUMBER() OVER(PARTITION BY leaf_amka ORDER BY

level DESC) για να κατατάξουμε τις γραμμές κάθε γιατρού και, μέσω του εξωτερικού φιλτραρίσματος WHERE number = 1, απομονώνουμε αποκλειστικά την τελική εγγραφή της αναδρομής που αποτυπώνει την πλήρη, ολοκληρωμένη διαδρομή μέχρι την κορυφή της ιεραρχικής δομής.

Q.14

Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

```
WITH annual_diagnoses AS (  
    SELECT  
        i.category,  
        YEAR(a.admission_date) AS diag_year,  
        COUNT(*) AS total_incidents  
    FROM admissions a  
    JOIN icd10 i ON a.admission_diagnosis_code = i.icd10_code  
    GROUP BY i.category, diag_year  
    HAVING total_incidents >= 5  
)  
SELECT  
    curr.category AS Diagnosis_Category,  
    curr.diag_year AS Year_A,  
    next_yr.diag_year AS Year_B,  
    curr.total_incidents AS Frequency  
FROM annual_diagnoses curr  
JOIN annual_diagnoses next_yr  
    ON curr.category = next_yr.category  
    AND curr.diag_year = next_yr.diag_year - 1  
WHERE curr.total_incidents = next_yr.total_incidents  
ORDER BY Year_A DESC, Diagnosis_Category;
```

Ορίζουμε ένα CTE, το οποίο πραγματοποιεί INNER JOIN μεταξύ των πινάκων admissions και icd10 για να εξάγει το έτος εισαγωγής και να καταμετρήσει τον ετήσιο όγκο περιστατικών ανά ευρύτερη κατηγορία διάγνωσης. Με το HAVING total_incidents >= 5, φιλτράρουμε αρχικά το CTE κρατώντας μόνο τις εγγραφές που παρουσιάζουν επαρκή συχνότητα, αποκλείοντας μεμονωμένα ή σπάνια περιστατικά. Έπειτα, εκτελούμε self-join του CTE, συσχετίζοντας την τρέχουσα εγγραφή (curr) με την εγγραφή του επόμενου ημερολογιακού έτους (next_yr.diag_year - 1). Τέλος, μέσω του WHERE, απομονώνουμε τις περιπτώσεις εκείνες όπου ο αριθμός των εισαγωγών παρέμεινε απόλυτα σταθερός μεταξύ των δύο διαδοχικών ετών, και ταξινομούμε το τελικό αποτέλεσμα σε φθίνουσα χρονολογική σειρά και αλφαβητικά ανά κατηγορία.

Q.15

Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```
-- Q15: Κατανομή triage ανά επίπεδο επείγοντος με μέσο χρόνο αναμονής, ποσοστό νοσηλείας και
κατανομή παραπομπών ανά τμήμα
WITH level_stats AS (
    SELECT
        t.urgency_level,
        COUNT(*) AS total_per_level,
        ROUND(AVG(TIMESTAMPDIFF(MINUTE, t.arrival_time,
                                t.triage_time)), 1) AS avg_wait_minutes,
        ROUND(SUM(t.outcome = 'hospitalized') * 100.0
              / COUNT(*), 1) AS hospitalization_pct
    FROM triages t
    WHERE t.triage_time IS NOT NULL
    GROUP BY t.urgency_level
)
SELECT
    t.urgency_level AS level,
    ul.name AS level_name,
    COALESCE(d.name, 'Μη νοσηλευόμενος') AS department_name,
    COUNT(*) AS cases_in_dept,
    ls.total_per_level,
    ls.avg_wait_minutes,
    ls.hospitalization_pct
FROM triages t
JOIN urgency_levels ul ON t.urgency_level = ul.level
JOIN level_stats ls ON t.urgency_level = ls.urgency_level
LEFT JOIN admissions a ON t.admission_id = a.admission_id
LEFT JOIN departments d ON a.department_id = d.dept_id
WHERE t.triage_time IS NOT NULL
GROUP BY t.urgency_level, ul.name, d.name,
         ls.total_per_level, ls.avg_wait_minutes, ls.hospitalization_pct
ORDER BY t.urgency_level, cases_in_dept DESC;
```

Χρησιμοποιούμε Common Table Expression (CTE) με όνομα level_stats για να προ-υπολογίσουμε τα aggregate στατιστικά ανά επίπεδο επείγοντος: συνολικό πλήθος περιστατικών, μέσο χρόνο αναμονής μέσω AVG(TIMESTAMPDIFF(MINUTE, ...)), και ποσοστό περιστατικών που οδήγησαν σε νοσηλεία ως SUM(t.outcome = 'hospitalized') * 100 / COUNT(*). Στο κύριο query κάνουμε JOIN τον πίνακα triages με τον lookup urgency_levels για το ονομαστικό όνομα του επιπέδου, με το CTE level_stats για να φέρουμε τα προ-υπολογισμένα στατιστικά, και LEFT JOIN με τους πίνακες admissions και departments για να βρούμε το τμήμα παραπομπής (όπου υπάρχει). Με COALESCE στο όνομα τμήματος εμφανίζουμε «Μη νοσηλευόμενος» για τα triage που δεν κατέληξαν σε εισαγωγή. Ομαδοποιούμε ανά επίπεδο και τμήμα για να μετρήσουμε τα περιστατικά ανά συνδυασμό, και ταξινομούμε το result set ανά επίπεδο επείγοντος και φθίνον πλήθος περιστατικών ανά τμήμα.